Projektarbeit

Entwicklung und Validierung einer Software zur autonomen Steuerung und Lokalisierung des Projektfahrzeugs Nimbulus-E

vorgelegt von
Constantin Commerell
Computational Science and Engineering
Sarah Heil
Wirtschaftsmathematik

Studiengang
Computational Science and Engineering,
Wirtschaftsmathematik

Prüfer
Prof. Dr. Thomas Schmitz

Prüfungsdatum
11.09.2026

Inhaltsverzeichnis

1 Einleitung und Motivation **3**
1.1 Einleitung . . . 3
1.2 Vorarbeiten . . . 3
1.3 Aufgabenstellung und Herangehensweise . . . 4

2 Grundlagen und Theorie **5**
2.1 Hardware . . . 5
2.2 Software . . . 7

3 Softwareentwicklung **12**
3.1 Ziel und Funktionsweise der Lokalisierung . . . 13
3.2 Monte-Carlo-Lokalisation . . . 14
3.3 Endpunktkorrektur . . . 21

4 Zusammenfassung und Ausblick **29**
4.1 Zusammenfassung . . . 29
4.2 Ausblick . . . 30

A Code-Listings **32**
A.1 Trajektorien . . . 32
A.2 Endpunktkorrektur . . . 40
A.3 Autonome Fahrzeugsteuerung . . . 42

Abbildungsverzeichnis **57**

Tabellenverzeichnis **58**

Codeverzeichnis **58**

Literatur **59**

Kapitel 1

Einleitung und Motivation

1.1 Einleitung

Autonome Fahrzeuge gewinnen heute immer mehr an Bedeutung. Sie stellen ein wichtiges Gebiet in der Forschung und Entwicklung moderner Mobilitätssysteme dar. Ihr Einsatz beschränkt sich jedoch nicht nur auf den Straßenverkehr. Auch in der Logistik, Industrie oder Landwirtschaft kommen selbstfahrende Systeme zum Einsatz, die sich selbstständig und zuverlässig in ihrer Umgebung orientieren und bewegen sollen. Voraussetzung dafür ist, dass das Fahrzeug seine Umgebung erfassen, die eigene Position in dieser Umgebung bestimmen und, mithilfe dieser Informationen eine geeignete Bewegung ausführen kann.
Das motivierte zur Entwicklung des Nimbulus-E, ein autonomes Fahrzeug mit acht Freiheitsgraden (FHG). Alle vier Räder des Fahrzeugs sind unabhängig lenkbar und angetrieben. Verschiedene Sensoren, beispielsweise ein Lidar-Sensor, ermöglichen die Erfassung der Fahrzeugumgebung.

Dieser Bericht dokumentiert die Weiterentwicklung der autonomen Navigation, basierend auf den Ergebnissen vorheriger Arbeiten.

1.2 Vorarbeiten

Für das Projekt wurde das autonome Testfahrzeug, Nimbulus-E, entwickelt. Es ist eine Transportplattform im Maßstab 1:3 mit acht Freiheitsgraden, das stetig von verschiedenen Gruppen weiterentwickelt wurde. Das Ziel war es, dem Fahrzeug zu ermöglichen seine Umgebung selbstständig zu kartieren, eine Trajektorie zu planen und dieser autonom zu folgen. In Vorarbeiten wurde ein VESC-Motorcontroller für die Radnabenmotoren in Betrieb genommen. Des Weiteren wurden die Ansteuerung der Schrittmotorcontroller für die Lenkung und die Verkabelung zwischen zwei Arduino-Mega-Controllern, den Sensoren und der Sicherheitstechnik vorgenommen.

Auf dieser Basis wurden drei Softwarekomponenten entwickelt. Dazu gehören eine Xbox-Controller-Steuerung, mit der das Fahrzeug manuell durch eine unbekannte Umgebung gefahren werden kann, während ein Lidar-Sensor parallel Umgebungsscans auf-

nimmt. Diese Scans wurden mithilfe eines SLAM-Algorithmus zu einer 2D- Belegungskarte verarbeitet.
Eine autonome Fahrzeugsteuerung plant auf Basis dieser Karte mit dem Hybrid-A^*Algorithmus eine Trajektorie und rechnet für alle vier Räder in Lenkwinkel und Raddrehzahlen um. Dies geschieht auf einem Konzept des Momentanpols, welches auf Verfahren der inversen Kinematik basiert.
Zuletzt gehört zu den Softwarekomponenten ein Kalibrierungsprogramm für die Absolutwertgeber der Lenkwinkelsensoren. (vgl. [3])

1.3 Aufgabenstellung und Herangehensweise

Die Aufgabenstellung des Projekts bestand in der Weiterentwicklung der autonomen Navigation des Nimbulus-E. Eines der Ziele war eine Live-Lokalisierung des Fahrzeugs innerhalb einer vorgegebenen Karte zu ermöglichen. Mit Hilfe von Lidar-Scans, die während der Fahrt aufgenommen wurden, sollte die aktuelle Position und Ausrichtung des Fahrzeugs möglichst zuverlässig bestimmt werden. Abweichungen zwischen der geplanten Trajektorie und der tatsächlichen Position des Fahrzeugs sollen erkannt werden können. Diese Abweichungen können während der autonomen Fahrt unter anderem durch Bodenunebenheiten und der Trägheit des Fahrzeugs entstehen.

Für die Lokalisierung wurde eine Monte-Carlo-Lokalisation (3.2) verwendet. Dabei werden die aktuellen Lidar-Scans mit der bereits vorhandenen Karte kombiniert, um die aktuelle Position des Fahrzeugs möglichst präzise zu schätzen. Auf Grundlage dieser Schätzung sollte anschließend eine Korrektur der ursprünglichen Trajektorie ermöglicht werden.

Außerdem wurde die Endpunktkorrektur (3.3) entwickelt. Sie führt das Fahrzeug, ausgehend von seinem aktuellen Zustand zum ursprünglichen geplanten Endzustand. Die Korrektur setzt sich aus einer geradlinigen Bewegung zum gewünschten Endpunkt und einer anschließenden Rotation zusammen. Die Rotation erreicht die ursprünglich vorgesehene Endausrichtung des Fahrzeugs. Dadurch wird eine neue Trajektorie zum ursprünglich geplanten Endzustand erzeugt, ausgehend von der aktuellen Position des Fahrzeugs.

Kapitel 2

Grundlagen und Theorie

In diesem Kapitel werden die theoretischen Grundlagen zusammengefasst, die für das Verständnis der im Rahmen dieser Arbeit verwendeten Methoden und deren Umsetzung erforderlich sind. Dabei wird insbesondere auf die Vorarbeiten von Schmitz, Seika und Nonnenmacher zurückgegriffen, auf denen wesentliche Teile des bestehenden Systems basieren [3]. Die für die Weiterentwicklung relevanten Konzepte werden im Folgenden kurz erläutert. Dadurch wird die Grundlage für die anschließend beschriebene Implementierung und Erweiterung des autonomen Fahrzeugs geschaffen.

2.1 Hardware

2.1.1 Rahmen/Karosserie

Der Rahmen des Fahrzeugs ist eine Item-Profilkonstruktion aus Aluminiumstreben, die auf maximal 100 kg Zuladung ausgelegt ist. Dazu gehören diverse 3D-gedruckte Gehäuse für Motoren, Controller und Ultraschallsensoren. Eine Hartplastikbox am Heck dient als zentrale Steuereinheit. (vgl. [3], Abschnitt 2.1)

2.1.2 Lenkung/Schrittmotoren

Closed-Loop CL57T(V4.0)-Schrittmotorcontroller steuern NEMA-24-Schrittmotoren mit integriertem 1000-RPR-Encoder. Über ein Schneckengetriebe mit 18:1 Übersetzung entsprechen 200 Motorschritte einer vollen Umdrehung der Lenkachse. Dabei sind 10 Schritte etwa 1° Lenkwinkel. Bei allen vier Rädern bedeutet „Richtungspin HIGH“ eine mathematisch positive, also gegen den Uhrzeigersinn gerichtete, Raddrehung. Hier blinkt bei zu hohem Gegenmoment die rote Controller-LED siebenmal. Durch Nachdrehen der Kupplung bei ausgeschaltetem Fahrzeug kann dies behoben werden. (vgl. [3], Abschnitt 2.2.1)

2.1.3 Antrieb/Radnabenmotoren

Der Antrieb des Fahrzeugs erfolgt über vier Vbest Life 36 V 350 W Radnabenmotoren. Die Ansteuerung der Radnabenmotoren erfolgt über vier VESC-Controller. Arduino 1 kommuniziert dabei über eine UART-Verbindung mit dem Mastervesccontroller am hinteren linken Rad mit der ID 0. Die vier VESCs sind untereinander über einen CAN-Bus verbunden, sodass die Soll-Drehzahlen an die einzelnen Motorcontroller weitergegeben werden können. Zur softwareseitigen Ansteuerung wird die Klasse *VescNimbulus* verwendet. Diese berücksichtigt unter anderem die Polpaarzahl von 15 und übernimmt die Umrechnung zwischen ERPM (Umdrehung pro Minute) und RPS (Umdrehungen pro Sekunde).

Ein wichtiger Parameter bei der Ansteuerung der Motoren ist die minimale, durch die VESCs ansteuerbare Drehzahl. Diese ist aktuell auf 180 ERPM eingestellt, was ungefähr 0,2 RPS entspricht. Die Wahl dieses Wertes stellt einen Kompromiss dar. Bei einer zu niedrigen Einstellung können die Motoren im Stillstand zu zittern beginnen. Wird der Wert dagegen zu hoch gewählt, können insbesondere während der autonomen Fahrt Probleme auftreten, wenn geringe Soll-Drehzahlen benötigt werden. (vgl. [3], Abschnitt 2.2.2)

2.1.4 Elektronik/Verkabelung

Das Steuerungssystem des Fahrzeugs basiert auf zwei Arduino Mega. Arduino 1 über nimmt dabei einen Großteil der zentralen Aufgaben. Dazu gehören die Sicherheitsfunktionen, die Auswertung des Kompasses und der Absolutwertgeber, sowie die Ansteuerung der Radnabenmotoren über die VESCs. Zusätzlich stellt Arduino 1 die Kommunikation mit dem PC und damit mit MATLAB her. Arduino 2 ist dagegen ausschließlich für die Ansteuerung der Lenkwinkel über die Schrittmotoren zuständig. Die Stromversorgung des Fahrzeugs erfolgt über Lithium-Ionen Batterien von X-go. (vgl. [3], Abschnitt 2.2.3)

2.1.5 Sensorik

Absolutwertgeber Zur Erfassung des aktuellen Lenkwinkels befinden sich Absolutwertgeber an den Lenkwellen. Hierzu sind Magnete auf den Lenkwellen angebracht, deren Ausrichtung von den entsprechenden Sensoren erfasst wird. Bei der Kalibrierung zeigte sich, dass die beiden Arduinos trotz gleicher Versorgungsspannung geringfügig unterschiedliche ADC-Werte (Analog-to-Digital-Converter) liefern. Aus diesem Grund wurde die Kalibrierung für beide Mikrocontroller separat durchgeführt. Dafür wurden die Messwerte den entsprechenden Lenkwinkeln im Bereich von $-90°$ bis $+90°$ zugeordnet. (vgl. [3], Abschnitt 2.3.1)

Ultraschallsensoren, Kompass, Lidar Neben den Absolutwertgebern verfügt das Fahrzeug über weitere Sensorik. Insgesamt sind 16 HC-SR04-Ultraschallsensoren mit einer angegebenen Reichweite von 2 bis 300 cm und einer Genauigkeit von etwa drei

Millimetern verbaut. Diese sind derzeit jedoch noch nicht softwareseitig in die Fahrzeugsteuerung integriert (vgl. [3], Abschnitt 2.3.2). Zur Bestimmung der absoluten Orientierung wird ein Adafruit BNO055 als Kompass eingesetzt. Während der bisherigen Versuche konnten dabei keine relevanten langfristigen Driftfehler festgestellt werden (vgl. [3], Abschnitt 2.3.3). Für die Erfassung der Umgebung steht außerdem ein Velodyne Ultra-Puck VLP-32C als Lidar-Sensor zur Verfügung. Dieser besitzt 32 Kanäle, einen horizontalen Erfassungsbereich von 360° und einen vertikalen Erfassungsbereich von 40°. Die maximale Reichweite beträgt 200 Meter bei einer angegebenen Genauigkeit von etwa drei Zentimetern (vgl. [3], Abschnitt 2.3.4).

2.1.6 Sicherheitsfunktionen

Für einen sicheren Betrieb verfügt das Fahrzeug zusätzlich über mehrere Sicherheitsmechanismen. An den Lenkungen sind insgesamt acht ME-8108-Rollenschalter als Endanschläge angebracht, die bei einem Lenkwinkel von etwa $-100°$ und $+100°$ auslösen. Alle acht Endanschläge sind in Reihe geschaltet und werden mit 5 V betrieben. Wird einer der Endanschläge betätigt, löst ein Interrupt auf Arduino 1 eine Notstopp-Routine aus. Dabei werden die Radnabenmotoren gestoppt und die Räder anschließend wieder in ihre Nullposition gebracht. Zusätzlich ist der zulässige Lenkwinkel softwareseitig auf 95° begrenzt. Trotzdem kam es während der Versuche vereinzelt zu Fehlansteuerungen der Schrittmotoren, deren genaue Ursache bisher nicht abschließend geklärt werden konnte.

Neben den Endanschlägen stehen ein Not-Aus-Knopf am Fahrzeug sowie ein Funk-Not-Aus zur Verfügung. Beide lösen eine Notstopp-Routine aus und stoppen die Motoren unmittelbar. Im Gegensatz zum Auslösen eines Endanschlags werden die Räder hierbei nicht auf 0° zurückgestellt, sondern verbleiben in ihrer aktuellen Position. Nach dem Betätigen des Not-Aus ist für die erneute Inbetriebnahme ein Neustart des Arduino-Codes erforderlich. (vgl. [3], Abschnitt 2.4)

2.2 Software

2.2.1 Xbox-Controller-Steuerung

Für die manuelle Steuerung des Fahrzeugs wurde eine Xbox-Controller-Steuerung implementiert. Diese ermöglicht sowohl eine rein translatorische Bewegung, als auch eine Rotation des Fahrzeugs um seinen Schwerpunkt. Zusätzlich kann während der manuellen Fahrt die Umgebung mit Hilfe des Lidar-Sensors erfasst und kartiert werden. (vgl. [3], Abschnitt 3.1)

Die Kommunikation zwischen MATLAB und Arduino 1 erfolgt über eine serielle Schnittstelle. MATLAB übermittelt einen Steuerbefehl bestehend aus *steeringCommand*, *steeringAngle* und *rotRate*. Arduino 1 sendet im Gegenzug alle 500 Millisekunden die aktuellen Odometriedaten an MATLAB zurück. Für die Kommunikation der Lenkwinkel

zwischen Arduino 1 und Arduino 2 wird das Enum *steeringCmd* mit möglichen Zuständen *NO, YES, TURN, POS, NEG* und *END* verwendet. (vgl. [3], Abschnitt 3.1.1)

Die Fahrtrichtung wird über den linken Joystick des Xbox-Controllers vorgegeben, während die Geschwindigkeit über die Trigger *RT* und *LT* gesteuert wird. Dabei können Drehzahlen zwischen $0,15$ und $1,3$ RPS eingestellt werden. Mit der Taste *A* werden die Lenkwinkel auf ihre Ausgangsposition zurückgesetzt. Die Taste *Y* aktiviert den Drehmodus, bei dem die Räder auf Lenkwinkel $-57°$ und $+57°$ gestellt werden. Dadurch liegt der Momentanpol im Mittelpunkt des Fahrzeugs und eine Rotation um den eigenen Schwerpunkt wird ermöglicht. Die Drehrichtung kann anschließend über *RB*, beziehungsweise *LB* gewählt werden. Mit der Taste *X* wird das Programm beendet. Überschreitet ein berechneter Lenkwinkel $-90°$ oder $+90°$, wird dieser auf den entsprechenden Gegenwinkel umgerechnet und gleichzeitig die Drehrichtung des betroffenen Rades invertiert. (vgl. [3], Abschnitt 3.1.2)

Die während der manuellen Steuerung tatsächlich zurückgelegte Trajektorie wird anhand der vom Fahrzeug empfangenen Odometriedaten mithilfe inverser Kinematik rekonstruiert. Die Änderungen der x- und y-Position (Δx und Δy) werden aus den gemessenen Raddrehzahlen, dem Reifenradius und der aktuellen Orientierung ψ bestimmt. Die daraus berechnete Pose des Fahrzeugs kann anschließend als zusätzliche Posenschätzung für den SLAM-Algorithmus verwendet werden. (vgl. [3], Abschnitt 3.1.3)

2.2.2 SLAM-Algorithmus

Die Kartierung der Fahrzeugumgebung basiert auf einem bereits bestehenden SLAM-Ansatz (Simultaneous Localization And Mapping), der im Rahmen der vorherigen Projektarbeiten für die Online-Verarbeitung erweitert und angepasst wurde (vgl. [2]). Die grundlegende Aufbereitung der Lidar-Daten blieb dabei erhalten. Die aufgenommenen Scans werden zunächst gefiltert und anschließend von drei auf zwei Dimensionen projiziert, bevor sie dem SLAM-Algorithmus übergeben werden.

Eine wesentliche Erweiterung besteht darin, dass die Lidar-Scans direkt während der Fahrt verarbeitet werden. Um die dafür benötigte Rechenzeit zu reduzieren, wird die relative Bewegung des Fahrzeugs zwischen zwei Scans nicht ausschließlich aus den Lidar-Daten geschätzt. Stattdessen wird zusätzlich eine aus den Odometriedaten berechnete Posenschätzung bereitgestellt. Dadurch konnte die benötigte Rechenzeit gegenüber der alleinigen Schätzung aus den Scandaten deutlich reduziert werden.

Da die Verarbeitung der Lidar-Daten dennoch vergleichsweise rechenintensiv ist, wird die MATLAB Parallel Computing Toolbox eingesetzt. Die Aufbereitung und Verarbeitung der Scans erfolgt in einem separaten parallelen Prozess, während die eigentliche Fahrzeugsteuerung weiterhin ausgeführt wird. Die Lidar-Daten werden dabei in einem Intervall von fünf Sekunden verarbeitet. Dadurch soll verhindert werden, dass die rechenintensive Kartierung die manuelle Steuerung des Fahrzeugs beeinträchtigt.

Für die Kartierung wird *lidarSLAM* aus der MATLAB Navigation Toolbox verwendet. Die Erkennung von Schleifenschlüssen, sowie die Optimierung der Karte werden dabei

innerhalb des SLAM-Verfahrens durchgeführt. Zusätzlich kann die aus der Odometrie bestimmte Pose mit den Informationen der Lidar-Scans kombiniert werden, um eine genauere Schätzung der Fahrzeugbewegung zu erhalten. (vgl. [3], Abschnitt 3.1.4)

2.2.3 Autonome Fahrzeugsteuerung

Neben der manuellen Steuerung kann der Nimbulu-E eine zuvor geplante Trajektorie autonom abfahren. Hierfür stehen die Trajektorienarten *Circe, Rotating, Rotating and Moving*, als auch *From Occupation Map* zur Verfügung. Die ersten drei Varianten dienen insbesondere als Testtrajektorien zur Überprüfung der Fahrzeugsteuerung und der Umrechnung in Lenkwinkel und Raddrehzahlen. Hierzu gehören eine Kreisfahrt, eine periodische Rotation auf der Stelle, sowie eine Geradeausfahrt, die mit einer solchen Rotationsbewegung überlagert wird. Die Variante *From Occupation Map* ermöglicht die eigentliche Pfadplanung innerhalb einer zuvor erstellten Belegungskarte. (vgl. [3], Abschnitt 3.2)

Während der autonomen Fahrt wird die Kommunikation zwischen MATLAB und Arduino 1 über einen Timer-Interrupt gesteuert. In einem zeitlichen Abstand von 100 Millisekunden sendet Arduino 1 die aktuellen Fahrzeugdaten an MATLAB. Diese enthalten einen Zeitstempel, die Orientierung ψ, die vier gemessenen Lenkwinkel und die vier Raddrehzahlen. MATLAB verarbeitet die empfangenen Daten und sendet anschließend die Sollwerte der nächsten Trajektorienzeile an Arduino 1. Die Zeitschrittweite der Trajektorie wird dementsprechend auf diesen Interrupt abgestimmt. (vgl. [3], Abschnitt 3.2.1 und 3.2.2)

Für die Planung einer Trajektorie innerhalb einer realen Umgebung wird der „Hybrid-A^*“-Algorithmus aus der MATLAB Navigation Toolbox verwendet. Zunächst wird eine zuvor erstellte Belegungskarte geladen und anhand eines experimentell bestimmten Schwellenwerts binarisiert. Anschließend wird unter Berücksichtigung der fahrzeugspezifischen Parameter ein *plannerHybridAStar*-Objekt erzeugt und eine Trajektorie vom gewünschten Start- zum Zielzustand berechnet. Der„Hybrid-A^*“-Algorithmus ermöglicht dabei unter anderem Rückwärtsfahrten und enge Wendemanöver unter Berücksichtigung eines vorgegebenen Mindestwenderadius. Die berechnete Trajektorie wird anschließend mit *optimizePath* geglättet. Aus den Trajektoriendaten werden die benötigten Geschwindigkeiten bestimmt und schließlich auf die Interruptfrequenz von 10 Hz interpoliert. (vgl. [3], Abschnitt 3.2.3)

2.2.4 Umrechnung der Trajektorie in Lenkwinkel und Raddrehzahlen

Damit eine geplante Trajektorie vom Fahrzeug abgefahren werden kann, müssen die Bewegungsgrößen des Fahrzeugschwerpunkts in die Sollwerte der vier einzelnen Räder umgerechnet werden. Grundlage hierfür ist die Annahme nicht-holonomer Bindungen und damit eines schlupffreien Rollens der Räder.

Für die Berechnung wird das Konzept des Momentanpols verwendet. Dieser beschreibt den Punkt, um den sich das Fahrzeug zu einem bestimmten Zeitpunkt scheinbar rein rotatorisch bewegt. Aus der Geschwindigkeit ν_S des Fahrzeugschwerpunkts, der Drehgeschwindigkeit $\dot{\psi}$ sowie den bekannten geometrischen Verbindungsvektoren zwischen Fahrzeugschwerpunkt und den einzelnen Rädern können die jeweiligen Radgeschwindigkeiten bestimmt werden. Hierfür wird die Beziehung $\nu_Q = \nu_S + \dot{\psi} \times_S r_Q$ verwendet. Aus den resultierenden Geschwindigkeitsvektoren werden dann die benötigten Lenkwinkel und Raddrehzahlen berechnet. Die Lenkwinkel ergeben sich mit Hilfe des Arkustangens. Um unerwünschte Sprünge zwischen äquivalenten Winkeldarstellungen zu verhindern, wird die eigens entwickelte Funktion *findNearestTan* eingesetzt. Diese wählt denjenigen möglichen Winkel, der dem zuvor eingestellten Lenkwinkel am nächsten liegt. Die benötigte Raddrehzahl ergibt sich aus dem Zusammenhang $\nu = 2\pi nr$. Das Vorzeichen und damit die Fahrtrichtung des jeweiligen Rades wird anhand des Skalarprodukts zwischen dem Lenkwinkel- und dem Geschwindigkeitsvektor bestimmt. (vgl. [3], Abschnitt 3.2.4)

Auch während der autonomen Fahrt wird inverse Kinematik eingesetzt um aus den gemessenen Raddaten die tatsächliche Fahrzeugbewegung zu rekonstruieren. Im Gegensatz zur reinen Controllersteuerung können bei der autonomen Fahrt Translation und Rotation gleichzeitig auftreten, wodurch die Berechnung aufwendiger wird. Aus jedem der vier Räder kann eine separate Schätzung für die Geschwindigkeit des Fahrzeugschwerpunkts bestimmt werden. Da die Messwerte eines Radnabenmotors aufgrund defekter Hallsensoren zeitweise unzuverlässig waren, wurde zur robusteren Bestimmung nicht ausschließlich ein Mittelwert verwendet. Stattdessen werden die vier Schätzwerte sortiert und die beiden mittleren Werte gemittelt. Dadurch wird der Einfluss einzelner Ausreißer reduziert. (vgl. [3], Abschnitt 3.2.5)

2.2.5 Kalibrierung der Absolutwertgeber

Für die Kalibrierung der Absolutwertgeber wurde das separate Arduino-Programm *oneWheelSteering.ino* verwendet. Dieses ermöglicht es, einzelne Räder über den seriellen Monitor gezielt um eine vorgegebene Anzahl an Schritten zu bewegen. Gleichzeitig werden kontinuierlich die gemessenen Sensorspannungen ausgegeben. Das ist insbesondere nach dem Ausbau oder einer Veränderung der Position eines Absolutwertgebers notwendig, da anschließend die Referenzwerte für die Lenkwinkel von -90ř und $+90$ř erneut bestimmt werden müssen.

Bei der Verwendung des Kalibrierprogramms ist besondere Vorsicht geboten, da keine softwareseitige Sicherheitsbegrenzung gegen zu große Lenkwinkel implementiert ist. Eine zu starke Verstellung der Räder kann daher zu einer mechanischen Belastung oder Beschädigung der Endanschläge, beziehungsweise der Verkabelung führen. (vgl. [3], Abschnitt 3.3)

2.2.6 Ergebnisse der Kartierung und autonomen Fahrt

Die Funktionsfähigkeit der Xbox-Controller-Steuerung und des SLAM-Verfahrens wurde von Nonnenmacher, Schmitz und Seika im Automotive Lab experimentell untersucht. Dabei konnte mithilfe der während der manuellen Fahrt aufgenommenen Lidar-Scans eine Belegungskarte der Umgebung erstellt werden. Es zeigte sich, dass die Qualität der Kartierung insbesondere von einer zuverlässigen Erkennung von Schleifenschlüssen und ausreichenden Überlappungen zwischen den aufgenommenen Scans abhängt. Insgesamt konnte eine für die anschließende Trajektorienplanung geeignete Karte erzeugt werden. Darüber hinaus wurde gezeigt, dass das Fahrzeug einer autonom geplanten Trajektorie grundsätzlich folgen kann. Abweichungen zwischen geplanter und tatsächlich gefahrener Trajektorie traten insbesondere zu Beginn der Fahrt auf und wurden unter anderem auf Bodenunebenheiten sowie die Trägheit des Fahrzeugs beim Anfahren zurückgeführt (vgl. [3], Kapitel 4). Diese Ergebnisse bilden die Grundlage für die in dieser Arbeit vorgenommenen Weiterentwicklungen des autonomen Fahrbetriebs.

Kapitel 3

Softwareentwicklung

Die in Kapitel 2 beschriebene autonome Fahrzeugsteuerung ermöglicht es dem Nimbulus-E, auf Grundlage einer vorhandenen Belegungskarte eine Trajektorie zu planen und diese anschließend autonom abzufahren. Die tatsächlich gefahrene Trajektorie entspricht jedoch nicht immer exakt der zuvor geplanten Trajektorie. Während der Fahrt können verschiedene Störeinflüsse, wie Bodenunebenheiten, Trägheitseffekte oder Ungenauigkeiten bei der Ansteuerung der Räder zu Abweichungen führen. Insbesondere bei Fahrversuchen im Freien zeigte sich, dass bereits kleinere Unebenheiten oder Hindernisse die Bewegung des Fahrzeugs beeinflussen können.

Für eine zuverlässige autonome Navigation ist es daher notwendig, nicht ausschließlich von der ursprünglich geplanten Bewegung des Fahrzeugs auszugehen, sondern dessen tatsächlichen Zustand während und nach der Fahrt möglichst genau bestimmen zu können. Die bereits vorhandene Odometrie ermöglicht eine Rekonstruktion der gefahrenen Trajektorie anhand der gemessenen Lenkwinkel, Raddrehzahlen und Orientierung des Fahrzeugs. Zusätzlich sollte untersucht werden, ob die Position des Fahrzeugs unabhängig davon anhand seiner Umgebung bestimmt werden kann. Hierfür bietet sich der bereits zur Kartierung eingesetzte Lidar-Sensor an. Durch den Vergleich aktueller Lidar-Scans mit einer zuvor aufgenommenen Karte soll die aktuelle Pose des Fahrzeugs, bestehend aus Position und Orientierung, während der Fahrt geschätzt werden.

Im Rahmen unseres Projekts wurde dazu eine Live-Lokalisation auf Grundlage der Monte-Carlo-Localisation implementiert und in mehreren Fahrversuchen untersucht. Ein besonderer Schwerpunkt lag auf der Wahl geeigneter Parameter sowie dem Verhältnis zwischen Lokalisierungsgenauigkeit und dem dafür erforderlichen Rechenaufwand. Die durchgeführten Versuche zeigten, dass eine Lokalisierung des Fahrzeugs innerhalb der vorhandenen Karte grundsätzlich möglich ist. Unter den gegebenen Bedingungen konnte jedoch keine ausreichende Genauigkeit für die darauf aufbauende Korrektur der Fahrzeugposition erreicht werden.

Neben der Live-Lokalisation wurde deshalb eine Endpunktkorrektur auf Grundlage der Odometriedaten entwickelt. Diese soll Abweichungen zwischen dem tatsächlichen und dem geplanten Endzustand ausgleichen. Dazu wird ausgehend von der ermittelten Ist-Pose eine zusätzliche Trajektorie erzeugt, durch die zunächst die gewünschte Endposition und anschließend die vorgesehene Endausrichtung erreicht werden sollen.

Die beiden folgenden Abschnitte beschreiben die Entwicklung, Implementierung und Untersuchung der Monte-Carlo-Lokalisation, sowie der Endpunktkorrektur.

3.1 Ziel und Funktionsweise der Lokalisierung

Für die autonome Navigation eines Fahrzeugs ist neben der Erfassung der Umgebung und der Planung einer geeigneten Trajektorie die Kenntnis der aktuellen Fahrzeugpose von Bedeutung. Die Pose beschreibt den Zustand des Fahrzeugs innerhalb eines festgelegten Koordinatensystems und setzt sich im zweidimensionalen Fall aus der Position in x- und y-Richtung sowie Orientierung ψ zusammen.

Unter Lokalisierung wird in diesem Zusammenhang die Bestimmung dieser Pose innerhalb einer bereits bekannten Umgebung verstanden. Im Gegensatz zum zuvor beschriebenen SLAM-Verfahren muss dabei keine neue Karte erstellt werden. Stattdessen liegt eine Belegungskarte der Umgebung bereits vor und dient als Referenz für die Positionsbestimmung des Fahrzeugs.

Eine Möglichkeit zur Bestimmung der Fahrzeugpose stellt die in Kapitel 2 beschriebene Odometrie dar. Hierbei wird die Bewegung des Fahrzeugs anhand der gemessenen Raddrehzahlen, Lenkwinkel und Orientierung rekonstruiert. Ausgehend von einer bekannten Startpose kann dadurch die jeweils folgende Pose des Fahrzeugs berechnet werden. Die Positionsbestimmung beruht dabei jedoch auf der tatsächlich gemessenen, beziehungsweise daraus berechneten Bewegung des Fahrzeugs und nicht auf einem direkten Vergleich mit seiner Umgebung. Abweichungen in den zugrunde liegenden Messgrößen können sich daher auf die berechnete Position auswirken.

Das Ziel der Live-Lokalisation war es deshalb, zusätzlich die vom Lidar-Sensor erfasste Umgebung für die Positionsbestimmung zu verwenden. Während der Fahrt werden aktuelle Lidar-Scans aufgenommen, welche die Abstände des Fahrzeugs zu Objekten und Strukturen in seiner Umgebung erfassen. Da die Umgebung bereits in Form einer Belegungskarte bekannt ist, können diese Messungen mit der Karte in Beziehung gesetzt werden. Vereinfacht betrachtet wird dabei untersucht, an welcher Position und mit welcher Orientierung innerhalb der Karte die aktuell gemessenen Strukturen zu erwarten wären.

Für diese Lokalisierung müssen sowohl die Bewegung des Fahrzeugs, als auch die aktuellen Sensormessungen berücksichtigt werden. Die Odometriedaten liefern eine Schätzung darüber, wie sich das Fahrzeug seit der vorherigen Positionsbestimmung bewegt hat. Die Lidar-Daten stellen dagegen Informationen über die tatsächlich erfasste Umgebung zur Verfügung. Werden diese beiden Informationen kombiniert, kann die aktuelle Pose des Fahrzeugs innerhalb der bekannten Karte geschätzt werden.

Für die Umsetzung dieser Live-Lokalisation haben wir die Monte-Carlo-Lokalisation verwendet. Dabei wird die mögliche Fahrzeugpose nicht ausschließlich durch eine einzelne Position beschrieben, sondern es werden mehrere mögliche Posen betrachtet und

anhand der aufgenommenen Sensordaten bewertet. So kann schrittweise eine Schätzung der wahrscheinlichsten Fahrzeugpose bestimmt werden.

3.2 Monte-Carlo-Lokalisation

Die Monte-Carlo-Lokalisation (MCL) ist ein probabilistisches Verfahren zur Bestimmung der Pose eines mobilen Systems innerhalb einer bekannten Umgebung. Das Verfahren basiert auf einem Partikelfilter und berücksichtigt, dass die tatsächliche Position und Orientierung eines Fahrzeugs aufgrund von Messungenauigkeiten und Unsicherheiten in der Fahrzeugbewegung nicht exakt bekannt sind. Anstatt zu jedem Zeitpunkt nur eine einzelne Pose als tatsächlichen Fahrzeugzustand anzunehmen, wird deshalb eine Wahrscheinlichkeitsverteilung über verschiedene mögliche Zustände betrachtet.

Bei der MCL wird diese Wahrscheinlichkeitsverteilung durch eine endliche Menge sogenannter Partikel angenähert. Jedes Partikel stellt dabei eine mögliche Pose des Fahrzeugs dar und enthält dementsprechend eine Position in x- und y-Richtung, sowie eine Orientierung ψ. Ein einzelnes Partikel kann damit durch

$$\mathbf{x}^i = \begin{pmatrix} x^i \\ y^i \\ \psi^i \end{pmatrix} \tag{3.1}$$

beschrieben werden. Die Gesamtheit aller Partikel stellt verschiedene Hypothesen darüber dar, an welcher Stelle der Karte sich das Fahrzeug zu einem bestimmten Zeitpunkt befinden könnte. Zusätzlich wird den einzelnen Partikeln ein Gewicht zugeordnet, welches beschreibt, wie wahrscheinlich die jeweilige angenommene Fahrzeugpose unter Berücksichtigung der aktuellen Sensormessungen ist.

Die Partikelverteilung wird während der Fahrt fortlaufend an die Bewegung des Fahrzeugs und die neu aufgenommenen Sensordaten angepasst. Hierzu besteht ein Aktualisierungsschritt der Monte-Carlo-Lokalisation im Wesentlichen aus drei aufeinanderfolgenden Vorgängen. Zunächst werden die möglichen Fahrzeugposen entsprechend der seit der letzten Aktualisierung erfolgten Bewegung verschoben. Anschließend wird mithilfe der aktuellen Sensormessungen bewertet, wie gut die einzelnen angenommenen Posen mit der bekannten Umgebung übereinstimmen. Abschließend werden auf Grundlage dieser Bewertung neue Partikel ausgewählt. Partikel mit einer hohen Wahrscheinlichkeit werden dabei bevorzugt, während Partikel mit einer geringeren Wahrscheinlichkeit zunehmend verworfen werden. Dieser Vorgang wird als Resampling bezeichnet. (vgl. [1], Kapitel 4)

Durch die wiederholte Durchführung dieser Schritte konzentrieren sich die Partikel im Idealfall zunehmend auf den Bereich der Karte, in dem sich das Fahrzeug tatsächlich befindet. Aus der resultierenden Partikelverteilung kann schließlich eine Schätzung der aktuellen Fahrzeugpose bestimmt werden. Die Monte-Carlo-Lokalisation kombiniert somit Informationen über die Bewegung des Fahrzeugs mit den aktuellen Sensormessungen und der bereits bekannten Karte der Umgebung. (vgl. [1], Abschnitt 4.2)

3.2.1 Grundlagen der Monte-Carlo-Lokalisation

Die Aktualisierung der Partikel erfolgt zunächst auf Grundlage der Bewegung des Fahrzeugs. Hierfür wird ein Bewegungsmodell verwendet, welches beschreibt, wie sich die möglichen Fahrzeugposen aufgrund der seit der letzten Aktualisierung zurückgelegten Bewegung verändern. Da die Bewegung eines realen Fahrzeugs nicht vollständig fehlerfrei bestimmt werden kann, wird dabei eine gewisse Unsicherheit berücksichtigt. Die Partikel werden daher nicht ausschließlich um eine identische Bewegung verschoben, sondern entsprechend des Bewegungsmodells verteilt. Auf diese Weise entstehen nach der Bewegung mehrere leicht unterschiedliche Hypothesen für die neue Fahrzeugpose.

Nach dem Bewegungsupdate werden die aktuellen Sensormessungen verwendet, um die verschiedenen Positionshypothesen zu bewerten. Für die Lokalisierung des Nimbulus-E werden hierzu die Messungen des Lidar-Sensors herangezogen. Ein Lidar-Scan enthält Informationen über die Abstände zu den in der Umgebung erfassten Objekten. Da die Umgebung bereits in Form einer Belegungskarte bekannt ist, kann für jedes Partikel überprüft werden, wie gut die gemessenen Abstände mit den zu erwartenden Messungen an der jeweiligen angenommenen Pose übereinstimmen. Stimmen die Lidar Messungen gut mit den Strukturen der Karte überein, wird die entsprechende Fahrzeugpose als wahrscheinlicher bewertet. Das zugehörige Partikel erhält dementsprechend ein höheres Gewicht. Bei einer schlechten Übereinstimmung wird dem Partikel ein geringeres Gewicht zugeordnet.

Auf Grundlage dieser Gewichtung erfolgt anschließend das sogenannte Resampling. Dabei wird aus der bisherigen Partikelmenge eine neue Menge von Partikeln erzeugt. Partikel mit einem hohen Gewicht besitzen eine höhere Wahrscheinlichkeit, für die neue Partikelmenge ausgewählt zu werden. Dadurch können besonders wahrscheinliche Posen mehrfach vertreten sein, während Partikel mit einem geringen Gewicht mit der Zeit aus der Verteilung verschwinden. Die Partikelverteilung konzentriert sich dadurch zunehmend auf diejenigen Bereiche der Karte, die sowohl mit der Bewegung des Fahrzeugs als auch mit den aufgenommenen Sensordaten vereinbar sind. (vgl. [1])

Der Ablauf aus Bewegungsupdate, Bewertung anhand der Sensordaten und Resampling wird während der Lokalisierung wiederholt durchgeführt. Dabei ist die Anzahl der verwendeten Partikel von besonderer Bedeutung. Eine größere Partikelmenge ermöglicht es, mehr mögliche Fahrzeugposen gleichzeitig abzubilden, erhöht jedoch auch den für die Berechnungen erforderlichen Aufwand. Die Wahl einer geeigneten Partikelzahl stellt daher einen Kompromiss zwischen der Qualität der Approximation und dem benötigten Rechenaufwand dar. Dieser Zusammenhang war auch für die praktische Umsetzung der Monte-Carlo-Lokalisation am Nimbulus-E von Bedeutung und wird im Rahmen der Parametrisierung näher untersucht.

3.2.2 Umsetzung am Nimbulus-E

Für die Umsetzung der Monte-Carlo-Lokalisation wurde die bereits bestehende autonome Fahrzeugsteuerung in MATLAB erweitert. Die Lokalisierung kann dabei über die

Variable `useLidar` aktiviert werden. Zu Beginn wird die zuvor aufgenommene Belegungskarte geladen, innerhalb derer die aktuelle Pose des Fahrzeugs bestimmt werden soll. Anschließend wird mit `monteCarloLocalization` ein Objekt für die Durchführung der Monte-Carlo-Lokalisation erzeugt. Während der Fahrt werden dem MCL-Objekt die aus den Odometriedaten bestimmte Fahrzeugpose sowie die aktuellen Lidar-Messungen übergeben [5].

Für die Bewertung der Lidar-Messungen wird ein `likelihoodFieldSensorModel` verwendet [4]. Diesem wird die geladene Belegungskarte als Referenz übergeben. Der für die Lokalisierung verwendete Messbereich des Lidar-Sensors wird auf Entfernungen zwischen 0,2 und 10 Metern begrenzt. Da der Ursprung des Lidar-Sensors im verwendeten Modell mit dem Ursprung des Fahrzeugkoordinatensystems gleichgesetzt wird, wird die relative Sensorpose mit $[0, 0, 0]$ angegeben. Anschließend wird das konfigurierte Sensormodell der MCL zugewiesen.

Für die MCL wird eine `initialPose` von [0,0,0] vorgegeben. Die Unsicherheit dieser Ausgangsschätzung wird durch die Anfangskovarianz

$$\Sigma_0 = \mathrm{diag}(0{,}05,\ 0{,}05,\ 0{,}1) \tag{3.2}$$

berücksichtigt.
Darüber hinaus wird die zulässige Partikelzahl durch `ParticleLimits = [500,5000]` begrenzt. Eine Aktualisierung der Positionsschätzung erfolgt in Abhängigkeit von den festgelegten Bewegungsschwellwerten `UpdateThresholds = [0.05,0.05,0.01]`.

Die Odometriepose, die für die MCL benötigt wird, wird während der Fahrt fortlaufend aus den vom Fahrzeug übertragenen Messdaten bestimmt. Arduino 1 übermittelt hierzu unter anderem die vier aktuellen Lenkwinkel, die Drehzahlen der vier Radnabenmotoren sowie die vom Kompass bestimmte Orientierung und Gierrate des Fahrzeugs. Mithilfe der Funktion `odometry8DOFtoRelPose` wird aus diesen Messgrößen die innerhalb eines Zeitschritts zurückgelegte Verschiebung des Fahrzeugschwerpunkts bestimmt. Hierzu wird zunächst für jedes der vier Räder aus der gemessenen Drehzahl, dem Lenkwinkel und der Gierrate eine Schätzung der Geschwindigkeit des Fahrzeugschwerpunkts im Fahrzeugkoordinatensystem berechnet. Anschließend werden die vier Geschwindigkeitsschätzungen mithilfe der aktuellen Orientierung des Fahrzeugs in das Weltkoordinatensystem transformiert.

Da jedes der vier Räder eine eigene Schätzung der Schwerpunktgeschwindigkeit liefert, stehen für beide Geschwindigkeitskomponenten jeweils vier Werte zur Verfügung. Um den Einfluss einzelner abweichender Messwerte zu reduzieren, wird komponentenweise der Median der vier Geschwindigkeitsschätzungen gebildet. Durch Multiplikation mit der seit dem vorherigen Zeitschritt vergangenen Zeit Δt werden daraus die Verschiebungen Δx und Δy bestimmt. Diese werden über die einzelnen Zeitschritte aufsummiert und gemeinsam mit der gemessenen Orientierung zur aktuellen Odometriepose

$$\mathbf{x}_{Odom} = \begin{pmatrix} x_{Odom} \\ y_{Odom} \\ \psi_{Odom} \end{pmatrix} \tag{3.3}$$

zusammengefasst.

Parallel dazu werden die Messdaten des Lidar-Sensors eingelesen. Der Sensor liefert eine dreidimensionale Punktwolke, die mithilfe der MATLAB-Funktion `pc2scan` in einen zweidimensionalen `lidarScan` überführt wird [6]. Dadurch steht für die Monte-Carlo-Lokalisation ein zweidimensionaler Scan zur Verfügung, der mithilfe des zuvor konfigurierten Sensormodells mit der bekannten Belegungskarte in Beziehung gesetzt werden kann.

Die eigentliche Lokalisierung erfolgt durch die gemeinsame Übergabe der aktuellen Odometriepose und des zweidimensionalen Lidar-Scans an die MCL [5]. Als Ergebnis werden der Aktualisierungsstatus `isUpdated`, die geschätzte Fahrzeugpose `estimatedPose` sowie die zugehörige Kovarianz zurückgegeben. Wird eine Aktualisierung der Positionsschätzung durchgeführt, wird die neu bestimmte Pose gespeichert und kann anschließend mit der rein odometrisch bestimmten Fahrzeugpose verglichen werden.

3.2.3 Parametrisierung und Testfahrten

Für die praktische Anwendung der Monte-Carlo-Lokalisation wurden verschiedene Parameter experimentell an das Fahrzeug und die zur Verfügung stehende Rechenleistung angepasst. Einen wesentlichen Parameter stellt dabei die Anzahl der verwendeten Partikel dar. Eine größere Partikelmenge ermöglicht grundsätzlich eine genauere Approximation der Wahrscheinlichkeitsverteilung, erhöht jedoch gleichzeitig den erforderlichen Rechenaufwand [7]. Da die Lokalisierung während der autonomen Fahrt parallel zur Kommunikation mit dem Fahrzeug und zur Verarbeitung der Sensordaten erfolgt, musste daher ein geeigneter Kompromiss zwischen der Qualität der Lokalisierung und dem benötigten Rechenaufwand gefunden werden.

Die Wahl der Partikelgrenzen erfolgte experimentell anhand mehrerer Testfahrten. Als Ausgangskonfiguration wurde `ParticleLimits = [500,5000]` verwendet. Um den Einfluss der maximalen Partikelanzahl zu untersuchen, wurde diese sowohl reduziert als auch erhöht. Bei einer Verringerung auf `[500,2500]` zeigte die geschätzte Fahrzeugpose deutlich stärkere Schwankungen. Eine Erhöhung auf `[500,10000]` führte dagegen während der Versuche zu Beeinträchtigungen der autonomen Trajektorienabfahrt. Das Fahrzeug führte die vorgesehene Bewegung teilweise nicht mehr zuverlässig aus und kam in einzelnen Versuchen nicht wie vorgesehen zum Stillstand. Unter den untersuchten Konfigurationen erwies sich `[500,5000]` daher als der geeignetste Kompromiss und wurde für die weiteren Testfahrten verwendet.

Neben der Partikelanzahl wurden die `UpdateThresholds` variiert. Diese legen die Mindeständerung der Odometriepose fest, die erforderlich ist, bevor eine erneute Aktualisierung der MCL durchgeführt wird [5]. Dabei werden getrennte Schwellwerte für die translatorische Bewegung in x- und y-Richtung sowie für die Änderung der Orientierung vorgegeben. Durch experimentelle Variation wurde schließlich `UpdateThresholds = [0.05,0.05,0.01]` für die weiteren Versuche verwendet. Die endgültige Parametrisierung wurde somit anhand wiederholter Testfahrten und der dabei beobachteten Stabilität der Lokalisierung sowie des Fahrzeugverhaltens festgelegt.

Zur qualitativen Beurteilung der Lokalisierung wurden die geplante Trajektorie, die aus den Odometriedaten rekonstruierte Trajektorie und die durch die Monte-Carlo-Lokalisation geschätzte Trajektorie miteinander verglichen. Dabei folgte die MCL-Schätzung grundsätzlich dem Verlauf der odometrisch bestimmten Bewegung, wies jedoch stärkere Schwankungen auf. Sowohl die MCL- als auch die Odometrie-Trajektorie zeigten zudem Abweichungen von der geplanten Trajektorie.

Eine quantitative Bestimmung des Lokalisierungsfehlers war anhand dieser Versuche nicht möglich, da keine unabhängige Referenzmessung der tatsächlich gefahrenen Trajektorie zur Verfügung stand. Insbesondere kann die geplante Trajektorie nicht als Ground Truth verwendet werden, da auch die tatsächliche Fahrzeugbewegung von der Solltrajektorie abweichen kann. Die Bewertung der MCL erfolgte daher qualitativ anhand des Verlaufs der geschätzten Pose, des Vergleichs mit der Odometrie und des während der Testfahrten beobachteten Fahrzeugverhaltens.

Bei einer maximalen Partikelanzahl von 10000 wurde eine Beeinträchtigung der autonomen Fahrzeugsteuerung beobachtet. Als mögliche Ursache wird der erhöhte Rechenaufwand der Monte-Carlo-Lokalisation angenommen. Da die Verarbeitung der Lidar-Daten und die Lokalisierung während der laufenden Kommunikation zwischen MATLAB und Arduino erfolgen, könnte eine erhöhte Berechnungsdauer zu zeitlichen Verzögerungen innerhalb des Gesamtsystems geführt haben. Ein kausaler Zusammenhang zwischen der erhöhten Partikelanzahl und dem beobachteten Fahrverhalten konnte im Rahmen der durchgeführten Versuche jedoch nicht nachgewiesen werden.

3.2.4 Ergebnisse der Monte-Carlo-Lokalisation

Zur Bewertung der Monte-Carlo-Lokalisation wurden insgesamt vier Testfahrten durchgeführt. Dabei wurden zwei Kreisfahrten sowie zwei Fahrten entlang der Valley-Trajektorie betrachtet. Für jede Testfahrt werden die geplante Trajektorie, die aus den Fahrzeugdaten rekonstruierte Odometrie und die durch die Monte-Carlo-Lokalisation geschätzte Trajektorie gegenübergestellt. Zusätzlich werden die Trajektorien innerhalb der für die Lokalisierung verwendeten Umgebungskarte dargestellt. Da keine unabhängige Referenzmessung der tatsächlichen Fahrzeugpose zur Verfügung stand, dient der Vergleich in erster Linie der Beurteilung des Verlaufs und der Stabilität der Positionsschätzungen. Aus einer Abweichung von der geplanten Trajektorie kann nicht unmittelbar auf einen Lokalisierungsfehler geschlossen werden, da auch das reale Fahrzeug von der geplanten Bewegung abweichen kann.

Bei den beiden Kreisfahrten zeigt die Monte-Carlo-Lokalisation einen insgesamt gleichmäßigen und plausiblen Verlauf. Die grundlegende Kreisform wird in beiden Versuchen vollständig rekonstruiert. Wie in den Abbildungen 3.1 und 3.2 zu erkennen ist, verlaufen die MCL- und die Odometriekurve über weite Teile der Fahrt in einem ähnlichen Bereich, weisen jedoch lokale Unterschiede voneinander sowie von der geplanten Trajektorie auf. Die Abweichungen zwischen MCL und Odometrie verändern sich dabei im Verlauf der Fahrt. Dies ist mit der Funktionsweise der MCL vereinbar, da neben

der odometrisch bestimmten Bewegung auch die Übereinstimmung der aktuellen Lidar-Messungen mit der bekannten Umgebungskarte in die Positionsschätzung eingeht.

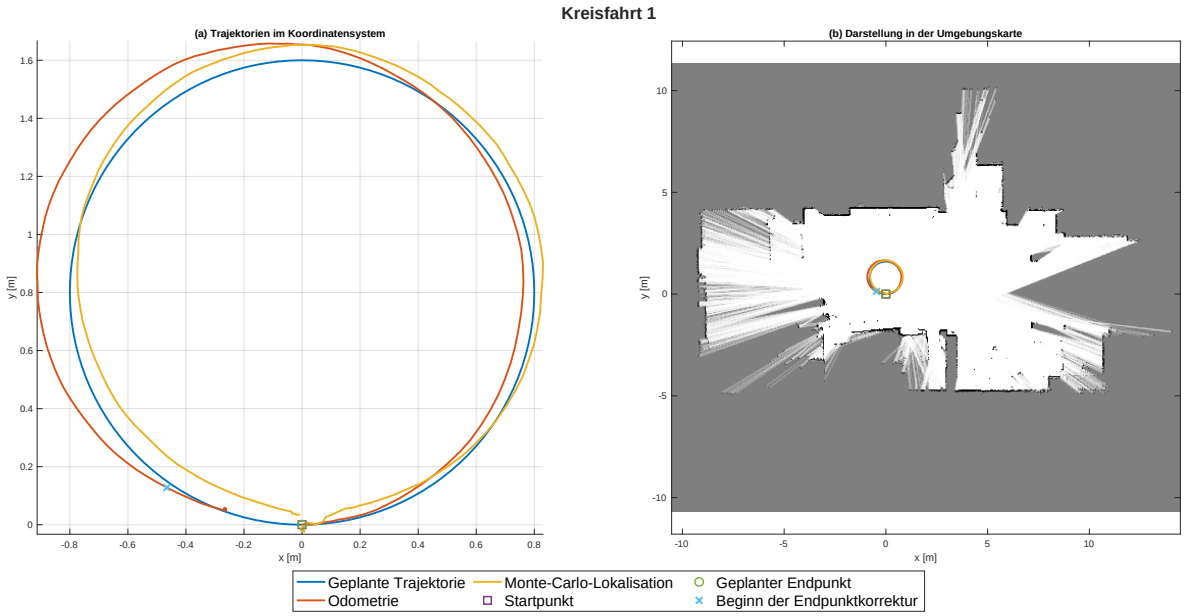


Abbildung 3.1: Vergleich der geplanten Trajektorie mit der Odometrie und der Monte-Carlo-Lokalisation für die erste Kreisfahrt sowie Darstellung der Trajektorien innerhalb der verwendeten Umgebungskarte.

Bei den beiden Valley-Trajektorien treten deutlich größere Unterschiede zwischen der geplanten und den geschätzten Trajektorien auf. Sowohl die Odometrie als auch die MCL bilden den charakteristischen Verlauf mit den geraden und gekrümmten Abschnitten grundsätzlich ab. Im mittleren Bereich der Fahrt verlaufen beide Positionsschätzungen jedoch deutlich unterhalb der geplanten Trajektorie. Gleichzeitig stimmen MCL und Odometrie nicht über den gesamten Verlauf miteinander überein. Insbesondere bei der ersten Valley-Trajektorie ist über längere Abschnitte ein sichtbarer Abstand zwischen den beiden Positionsschätzungen vorhanden.

Auffällig ist, dass sich die Ergebnisse der jeweils wiederholten Testfahrten qualitativ ähneln. Sowohl bei den beiden Kreisfahrten als auch bei den beiden Valley-Trajektorien zeigen sich vergleichbare Verläufe der Positionsschätzungen. Die bei den Valley-Trajektorien auftretenden größeren Abweichungen sind somit nicht ausschließlich auf einen einzelnen Versuch zurückzuführen.

Insgesamt zeigen die Testfahrten, dass die implementierte Monte-Carlo-Lokalisation grundsätzlich in der Lage ist, die Bewegung des Nimbulus-E innerhalb der zuvor aufgenommenen Umgebungskarte fortlaufend zu verfolgen. Gleichzeitig weist die MCL-Schätzung insbesondere bei den Valley-Trajektorien Schwankungen und Abweichungen gegenüber der Odometrie auf. Aufgrund der fehlenden unabhängigen Referenzmessung kann aus den vorliegenden Versuchen jedoch nicht bestimmt werden, welche der beiden Positionsschätzungen zu einem bestimmten Zeitpunkt näher an der tatsächlichen Fahrzeugpose liegt. Eine höhere Genauigkeit der MCL gegenüber der Odometrie konnte mit dem verwendeten Versuchsaufbau daher nicht nachgewiesen werden.

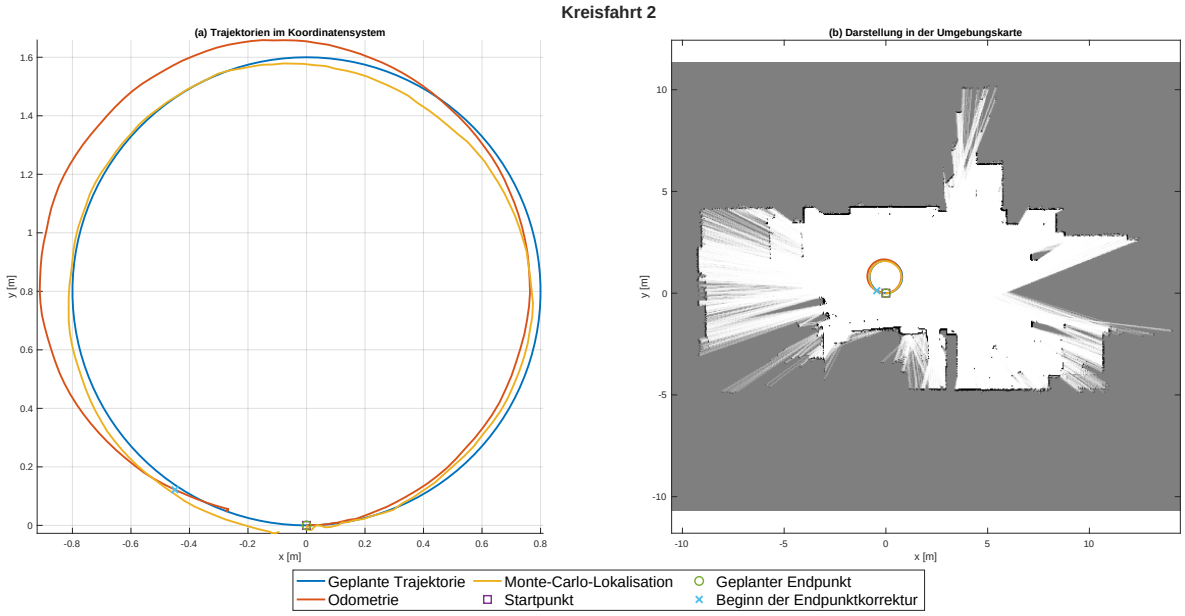


Abbildung 3.2: Vergleich der geplanten Trajektorie mit der Odometrie und der Monte-Carlo-Lokalisation für die zweite Kreisfahrt sowie Darstellung der Trajektorien innerhalb der verwendeten Umgebungskarte.

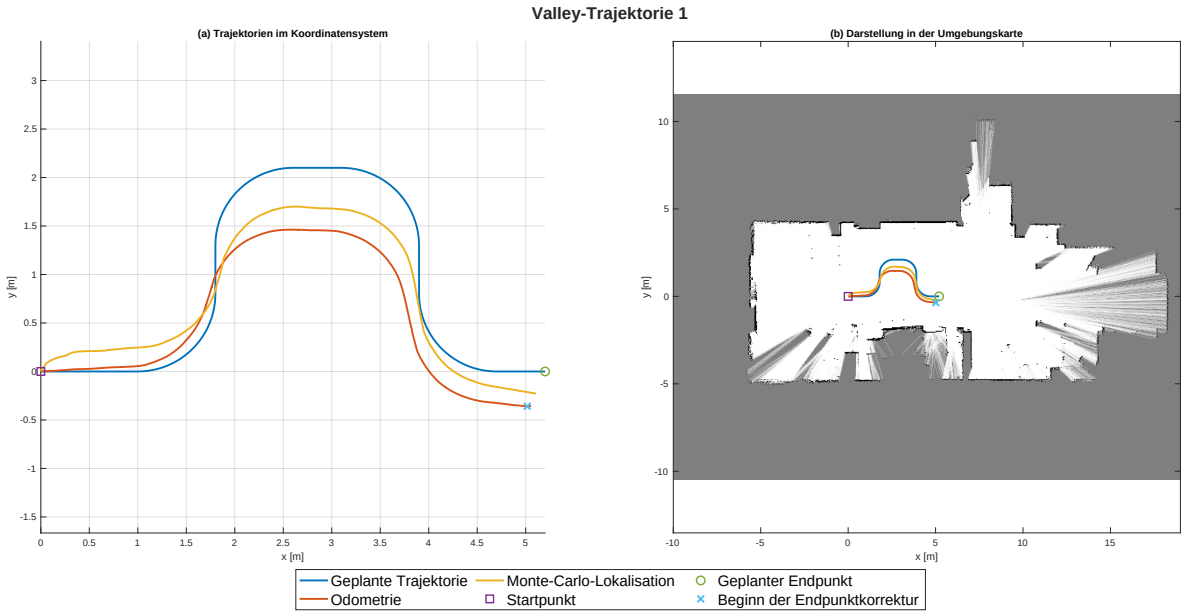


Abbildung 3.3: Vergleich der geplanten Trajektorie mit der Odometrie und der Monte-Carlo-Lokalisation für die erste Valley-Trajektorie sowie Darstellung der Trajektorien innerhalb der verwendeten Umgebungskarte.

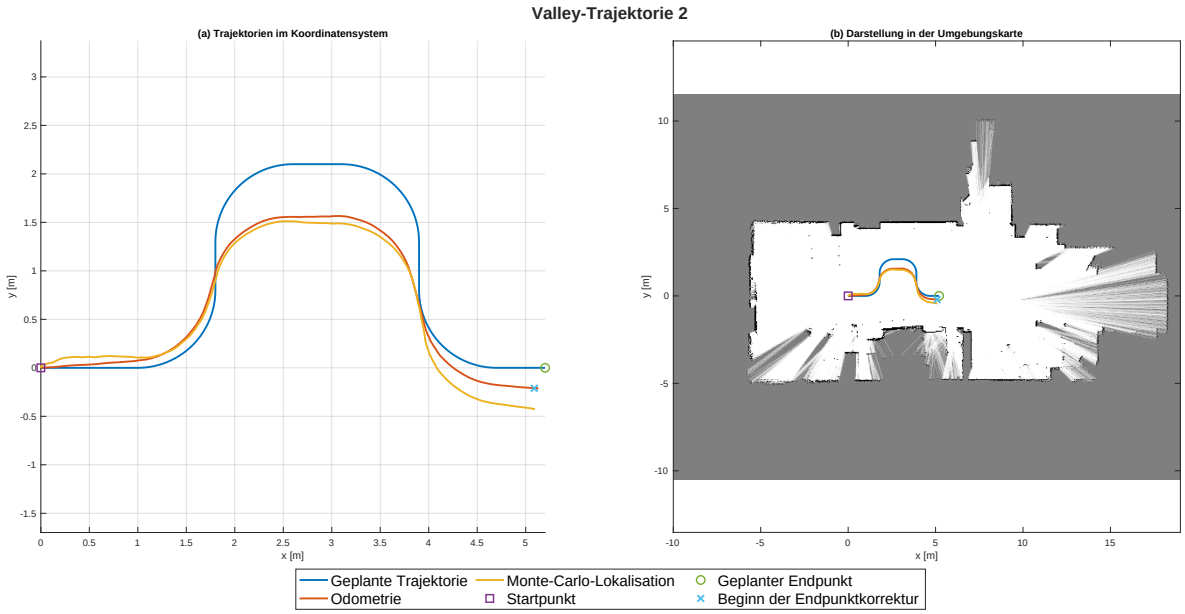


Abbildung 3.4: Vergleich der geplanten Trajektorie mit der Odometrie und der Monte-Carlo-Lokalisation für die zweite Valley-Trajektorie sowie Darstellung der Trajektorien innerhalb der verwendeten Umgebungskarte.

Für die anschließend entwickelte Endpunktkorrektur war eine möglichst stabile und zuverlässig verfügbare Schätzung der aktuellen Fahrzeugpose erforderlich. Da die odometrisch bestimmte Trajektorie in den durchgeführten Versuchen einen stabileren Verlauf zeigte und die Genauigkeit der MCL nicht unabhängig verifiziert werden konnte, wurde die MCL-Schätzung hierfür nicht verwendet. Die im folgenden Abschnitt beschriebene Endpunktkorrektur basiert daher auf der odometrisch bestimmten Fahrzeugpose.

3.3 Endpunktkorrektur

3.3.1 Ziel und Konzept der Endpunktkorrektur

Während der autonomen Trajektorienabfahrt kann die tatsächlich erreichte Endpose des Fahrzeugs von der ursprünglich geplanten Endpose abweichen. Ursachen hierfür können unter anderem Ungenauigkeiten bei der Ansteuerung der Räder, Bodenunebenheiten oder Trägheitseffekte des Fahrzeugs sein. Ziel der Endpunktkorrektur ist es daher, die nach Abschluss der ursprünglichen Trajektorie verbleibende Abweichung zwischen der aktuellen Fahrzeugpose und der vorgesehenen Endpose zu reduzieren.

Die Endpose setzt sich aus der Position des Fahrzeugs in x- und y-Richtung sowie seiner Orientierung ψ zusammen. Der gewünschte Endzustand kann somit durch

$$\mathbf{x}_{Ziel} = \begin{pmatrix} x_{Ziel} \\ y_{Ziel} \\ \psi_{Ziel} \end{pmatrix} \tag{3.4}$$

beschrieben werden. Nach Abschluss der ursprünglichen Trajektorie wird die aktuell erreichte Pose aus den Odometriedaten bestimmt und mit dem geplanten Endzustand verglichen. Die odometrische Pose wird, wie in Abschnitt 3.2.2 beschrieben, aus den während der Fahrt erfassten Fahrzeugdaten rekonstruiert. Die zuvor untersuchte Monte-Carlo-Lokalisation wird für die Endpunktkorrektur nicht verwendet.

Die Korrektur wird in zwei aufeinanderfolgende Schritte unterteilt. Zunächst wird die Position des Fahrzeugs korrigiert. Hierzu wird zwischen der aktuellen Position (x_{Ist}, y_{Ist}) und der gewünschten Endposition (x_{Ziel}, y_{Ziel}) eine geradlinige Korrekturtrajektorie erzeugt. Eine Besonderheit des Nimbulus-E besteht darin, dass aufgrund der unabhängig lenk- und antreibbaren Räder eine translatorische Bewegung nicht zwingend in Richtung der Fahrzeuglängsachse erfolgen muss. Die Orientierung des Fahrzeugs kann deshalb während dieser Positionskorrektur beibehalten werden, während sich das Fahrzeug entlang der direkten Verbindung zum gewünschten Endpunkt bewegt.

Nach dem Abfahren der translatorischen Korrekturtrajektorie erfolgt in einem zweiten Schritt die Korrektur der Orientierung. Das Fahrzeug wird hierzu auf der Stelle um seinen Mittelpunkt gedreht. Die notwendige Winkeländerung wird so bestimmt, dass die kürzere Drehrichtung zwischen der aktuellen und der gewünschten Orientierung gewählt wird. Während der Rotationsbewegung wird die aktuelle Orientierung fortlaufend überprüft und die Bewegung beendet, sobald die vorgegebene Winkeltoleranz erreicht ist.

Durch die Trennung von Positions- und Orientierungskorrektur kann die besondere Beweglichkeit des Fahrzeugs genutzt werden, während die Berechnung der Korrekturbewegung vergleichsweise einfach bleibt. Die Berechnung der translatorischen Korrekturtrajektorie sowie die anschließende Korrektur der Orientierung werden in den folgenden Abschnitten näher beschrieben.

3.3.2 Berechnung der Positionskorrektur

Nach Abschluss der ursprünglich geplanten Trajektorie wird zunächst die Abweichung zwischen der erreichten und der gewünschten Endposition bestimmt. Als aktuelle Fahrzeugpose wird dabei die zuletzt aus den Odometriedaten berechnete Pose verwendet. Die geplante Endpose wird aus dem letzten Punkt der ursprünglichen Trajektorie entnommen. Für die translatorische Positionskorrektur werden zunächst ausschließlich die x- und y-Koordinaten der beiden Posen berücksichtigt.

Aus der aktuellen Position $(x_{\mathrm{Ist}}, y_{\mathrm{Ist}})$ und der gewünschten Endposition $(x_{\mathrm{Ziel}}, y_{\mathrm{Ziel}})$ ergeben sich die verbleibenden Positionsabweichungen zu

$$\Delta x = x_{\mathrm{Ziel}} - x_{\mathrm{Ist}} \tag{3.5}$$

und

$$\Delta y = y_{\text{Ziel}} - y_{\text{Ist}}. \tag{3.6}$$

Der euklidische Abstand zwischen der aktuellen und der gewünschten Endposition beträgt damit

$$d = \sqrt{(\Delta x)^2 + (\Delta y)^2}. \tag{3.7}$$

Liegt dieser Abstand unterhalb einer festgelegten Toleranz von 0,05 m, wird keine translatorische Korrekturtrajektorie erzeugt. Positionsabweichungen von weniger als fünf Zentimetern werden damit für die weitere Korrektur nicht berücksichtigt.

Ist der Abstand größer oder gleich der vorgegebenen Toleranz, wird mit der Funktion `buildEndPointCorrectionTrajectory` eine geradlinige Korrekturtrajektorie von der aktuellen Position zur gewünschten Endposition erzeugt. Die Richtung der translatorischen Bewegung im Weltkoordinatensystem wird durch

$$\psi_{\text{Fahrt}} = \text{atan2}(\Delta y, \Delta x) \tag{3.8}$$

bestimmt. Dabei beschreibt ψ_{Fahrt} ausschließlich die Bewegungsrichtung des Fahrzeugschwerpunkts und nicht die Orientierung des Fahrzeugs. Während der translatorischen Korrektur wird die zu Beginn der Korrektur vorliegende Fahrzeugorientierung ψ_{Ist} konstant gehalten. Für die Orientierung und ihre Änderungsrate gilt somit

$$\psi(t) = \psi_{\text{Ist}}, \qquad \dot{\psi}(t) = 0. \tag{3.9}$$

Die translatorischen Geschwindigkeitskomponenten werden entsprechend der Bewegungsrichtung mit

$$\dot{x} = \nu_{\text{Korr}} \cos(\psi_{\text{Fahrt}}) \tag{3.10}$$

und

$$\dot{y} = \nu_{\text{Korr}} \sin(\psi_{\text{Fahrt}}) \tag{3.11}$$

bestimmt. Dadurch wird die Fähigkeit des Nimbulus-E genutzt, sich aufgrund seiner unabhängig lenk- und antreibbaren Räder in eine Richtung zu bewegen, die von der aktuellen Orientierung der Fahrzeuglängsachse abweicht.

Bei konstanter Korrekturgeschwindigkeit ν_{Korr} ergibt sich die vorgesehene Dauer der Korrekturbewegung aus

$$T_{\text{Korr}} = \frac{d}{\nu_{\text{Korr}}}. \tag{3.12}$$

Für die durchgeführten Versuche wurde eine Korrekturgeschwindigkeit von $\nu_{\text{Korr}} = 0{,}5\,\text{m/s}$ verwendet. Anhand der für die Fahrzeugsteuerung vorgegebenen Zeitschrittweite wird die Korrekturtrajektorie anschließend diskretisiert. Die Anzahl der Trajektorienpunkte wird dabei so gewählt, dass mindestens zwei Punkte erzeugt werden. Für jeden Punkt wird mit

$$s_i = \frac{i-1}{N-1}, \qquad i = 1, \ldots, N, \tag{3.13}$$

ein Interpolationsfaktor bestimmt. Die Positionen entlang der geradlinigen Verbindung ergeben sich damit zu

$$x_i = x_{\text{Ist}} + s_i \Delta x \tag{3.14}$$

und

$$y_i = y_{\text{Ist}} + s_i \Delta y. \tag{3.15}$$

Die erzeugte Korrekturtrajektorie besitzt mit

$$[\,t,\ x,\ y,\ \psi,\ \dot{x},\ \dot{y},\ \dot{\psi}\,] \tag{3.16}$$

dasselbe Datenformat wie die bereits für die autonome Trajektorienabfahrt verwendeten Trajektorien. Dadurch kann sie anschließend mit der bestehenden Funktion `trajectoryTo8DOF` in die benötigten Lenkwinkel und Raddrehzahlen der vier Räder umgerechnet werden. Eine separate Berechnung der Radkinematik für die Endpunktkorrektur ist somit nicht erforderlich.

Nach dem Abfahren der translatorischen Korrekturtrajektorie wird die Orientierung des Fahrzeugs in einem separaten Schritt korrigiert. Die Berechnung dieser Rotationsbewegung wird im folgenden Abschnitt beschrieben.

3.3.3 Korrektur der Endausrichtung

Nach der translatorischen Positionskorrektur wird die Orientierung des Fahrzeugs angepasst. Als Ausgangswinkel ψ_{Ist} wird hierfür die zu diesem Zeitpunkt aus den Fahrzeugdaten bestimmte Orientierung verwendet. Die Korrektur erfolgt als Rotation um den Fahrzeugmittelpunkt, wobei keine translatorische Bewegung vorgegeben wird.

Zur Erzeugung der Rotationsbewegung wird die Funktion`buildEndPointRotating2` verwendet. Aufgrund der Periodizität von Winkeln kann die Differenz zwischen aktueller und gewünschter Orientierung nicht ohne Berücksichtigung des Übergangs zwischen 0 und 2π bestimmt werden. Die für die Korrektur verwendete Winkeldifferenz wird daher durch

$$\Delta\psi = \operatorname{atan2}\left(\sin(\psi_{\text{Ziel}} - \psi_{\text{Ist}}), \cos(\psi_{\text{Ziel}} - \psi_{\text{Ist}})\right) \tag{3.17}$$

berechnet. Dadurch liegt $\Delta\psi$ im Bereich $[-\pi, \pi]$ und beschreibt die betragsmäßig kürzeste Winkeländerung zwischen der aktuellen und der gewünschten Orientierung.

Auf Grundlage dieser Winkeldifferenz wird eine Rotationstrajektorie erzeugt. Die translatorischen Geschwindigkeitskomponenten werden dabei zu

$$\dot{x} = 0, \qquad \dot{y} = 0 \tag{3.18}$$

gesetzt. Für die Winkelgeschwindigkeit wird abhängig von der erforderlichen Drehrichtung ein positiver oder negativer Wert mit dem Betrag

$$|\dot{\psi}| = 0{,}6\,\text{rad/s} \tag{3.19}$$

verwendet. Die Sollorientierung der einzelnen Trajektorienpunkte wird linear zwischen ψ_{Ist} und $\psi_{\text{Ist}}+\Delta\psi$ interpoliert. Die weitere Umrechnung in die Stellgrößen der vier Räder erfolgt analog zur zuvor beschriebenen Positionskorrektur mit `trajectoryTo8DOF`.

Für die durchgeführten Testfahrten wurde in der Fahrzeugsteuerung eine Zielorientierung von

$$\psi_{\text{Ziel}} = 2\pi \tag{3.20}$$

vorgegeben, was einer Orientierung von 360° beziehungsweise 0° entspricht. Die Funktion `buildEndPointRotating2` ist hingegen nicht auf diese Orientierung beschränkt, sondern verarbeitet die aktuelle und die gewünschte Orientierung als Eingangsparameter.

Während der tatsächlichen Rotationsbewegung wird die vom Fahrzeug gemessene Orientierung fortlaufend ausgewertet. Die Rotation wird beendet, sobald die Abweichung von der vorgegebenen Zielorientierung höchstens 5° beträgt. Zusätzlich ist die Anzahl der Schleifendurchläufe auf 100 begrenzt, um ein unbegrenztes Fortsetzen der Rotationsbewegung zu verhindern. Nach Beendigung der Schleife werden die Raddrehzahlen auf null gesetzt und die Endpunktkorrektur abgeschlossen.

3.3.4 Umsetzung in der autonomen Fahrt

Die Endpunktkorrektur wurde als nachgeschalteter Schritt in die bestehende autonome Fahrzeugsteuerung integriert. Nach dem vollständigen Abfahren der ursprünglich geplanten Trajektorie wird das Fahrzeug zunächst angehalten. Anschließend werden die geplante Endpose aus dem letzten Punkt der Solltrajektorie und die zu diesem Zeitpunkt odometrisch bestimmte Fahrzeugpose ermittelt. Auf Grundlage dieser beiden Posen wird die in Abschnitt 3.3.2 beschriebene translatorische Korrekturtrajektorie erzeugt.

Sofern eine translatorische Korrektur erforderlich ist, wird die erzeugte Trajektorie mit der bereits vorhandenen Funktion `trajectoryTo8DOF` in die benötigten Lenkwinkel und Raddrehzahlen der vier Räder umgerechnet. Dadurch kann für die Endpunktkorrektur dieselbe Verarbeitungskette wie für die ursprünglichen Solltrajektorien verwendet werden [3]. Vor Beginn der Bewegung werden zunächst die erforderlichen Lenkwinkel eingestellt, während die Raddrehzahlen noch null bleiben. Nach einer kurzen Wartezeit werden die berechneten Stellgrößen nacheinander an das Fahrzeug übertragen.

Während der Korrekturfahrt werden die aktuellen Fahrzeugdaten weiterhin eingelesen und zur Aktualisierung der odometrischen Pose verwendet. Die translatorische Korrekturtrajektorie wird während der Fahrt jedoch nicht erneut an die aktualisierte Fahrzeugpose angepasst. Sie wird einmalig aus der zu Beginn der Korrektur bestimmten Ist-Pose berechnet und anschließend abgefahren. Es handelt sich bei diesem Korrekturschritt somit nicht um eine kontinuierliche Regelung der Position.

Nach Abschluss der translatorischen Korrektur wird die aktuelle Orientierung erneut aus den Fahrzeugdaten bestimmt. Anschließend erfolgt die in Abschnitt 3.3.3 beschriebene Orientierungskorrektur. Auch für diese Bewegung werden zunächst die erforderlichen Lenkwinkel eingestellt. Während der anschließenden Rotation wird die gemessene Orientierung fortlaufend überprüft. Die Drehbewegung wird beendet, sobald die vorgegebene Winkeltoleranz erreicht oder die maximale Anzahl an Schleifendurchläufen überschritten wird. Abschließend werden die Raddrehzahlen auf null gesetzt.

Die entwickelte Endpunktkorrektur erweitert die bestehende autonome Fahrzeugsteuerung damit, ohne die vorhandene Trajektorienverarbeitung und Radkinematik grundlegend zu verändern. Als Grundlage für die Korrektur wird ausschließlich die odometrisch bestimmte Fahrzeugpose verwendet. Die Monte-Carlo-Lokalisation kann parallel weiterhin ausgeführt und aufgezeichnet werden, ihre Positionsschätzung fließt in der vorliegenden Implementierung jedoch nicht in die Endpunktkorrektur ein.

3.3.5 Ergebnisse der Endpunktkorrektur

Zur Untersuchung der Endpunktkorrektur wurden die bereits für die Bewertung der Monte-Carlo-Lokalisation verwendeten vier Testfahrten herangezogen. Dabei wurden zwei Kreisfahrten und zwei Fahrten entlang der Valley-Trajektorie durchgeführt. Die Kreisfahrten wurden mit einem Radius von $R = 0{,}8\,\mathrm{m}$ und einer Geschwindigkeit von $v_0 = 0{,}4\,\mathrm{m/s}$ ausgeführt. Für die Valley-Trajektorie wurden für die geraden Abschnitte

$v_{\text{gerade}} = 0{,}8\,\text{m/s}$ und für die Kurven $v_{\text{Kurve}} = 0{,}5\,\text{m/s}$ bei einem Radius von $R = 0{,}8\,\text{m}$ verwendet. Zwischen den einzelnen Trajektorienabschnitten war jeweils eine Pause von einer Sekunde vorgesehen.

Nach Abschluss der regulären Trajektorie wurde die odometrisch bestimmte Fahrzeugposition mit dem geplanten Endpunkt verglichen. Bei allen vier Testfahrten lag die daraus bestimmte Positionsabweichung oberhalb der für die Endpunktkorrektur festgelegten Toleranz von 0,05 m, sodass jeweils eine translatorische Korrektur durchgeführt wurde. Die vor Beginn der Korrektur bestimmten Abweichungen sind in Tabelle 3.1 dargestellt.

Tabelle 3.1: Odometrisch bestimmte Positionsabweichung vor Beginn der Endpunktkorrektur

Testfahrt	**Positionsabweichung**
Kreisfahrt 1	0,483 m
Kreisfahrt 2	0,466 m
Valley-Trajektorie 1	0,404 m
Valley-Trajektorie 2	0,238 m

Damit lag die odometrisch bestimmte Abweichung vor der Korrektur zwischen 0,238 m und 0,483 m. Die größten Abweichungen traten bei den beiden Kreisfahrten auf. Bei diesen entspricht der geplante Endpunkt nach einer vollständigen Umrundung wieder dem Startpunkt. Dieser wurde für die Versuche am Boden markiert, sodass die anschließende Korrekturbewegung zusätzlich visuell beurteilt werden konnte.

Nach Bestimmung der Positionsabweichung wurde die in Abschnitt 3.3.2 beschriebene Korrekturtrajektorie ausgeführt. Bei den Kreisfahrten konnte beobachtet werden, dass sich das Fahrzeug während der Korrektur wieder in Richtung des markierten Start- und damit Zielpunktes bewegte. Auch bei den beiden Valley-Trajektorien wurde eine Bewegung in Richtung des vorgesehenen Endpunktes beobachtet. Die Endpunktkorrektur führte damit in allen vier Versuchen die aus der odometrisch bestimmten Positionsabweichung berechnete Korrekturbewegung aus.

Eine quantitative Aussage über den tatsächlich verbleibenden Positionsfehler nach Abschluss der Korrektur ist mit dem verwendeten Versuchsaufbau jedoch nicht möglich. Die Bodenmarkierung diente lediglich der visuellen Kontrolle; der verbleibende Abstand zur tatsächlichen Zielposition wurde nicht durch ein unabhängiges Messsystem bestimmt. Eine allein aus der Odometrie berechnete Restabweichung wäre für die Bewertung der tatsächlichen Genauigkeit nur eingeschränkt aussagekräftig, da dieselbe odometrische Positionsschätzung bereits als Grundlage für die Berechnung der Korrekturtrajektorie verwendet wird. Aus den Versuchen kann daher gezeigt werden, dass die entwickelte Endpunktkorrektur die vorgesehene Korrekturbewegung ausführte und das Fahrzeug in Richtung des geplanten Endpunktes führte. Eine extern verifizierte Genauigkeit der erreichten Endposition kann daraus jedoch nicht abgeleitet werden.

Im Anschluss an die translatorische Korrektur wurde die Endausrichtung des Fahrzeugs entsprechend Abschnitt 3.3.3 korrigiert. Die Rotation wurde anhand der während der

Bewegung gemessenen Orientierung beendet, sobald die vorgegebene Winkeltoleranz erreicht war. Auch dieser Teil der Endpunktkorrektur konnte bei den durchgeführten Testfahrten ausgeführt werden. Aufgrund der verwendeten Toleranz sowie der beobachteten Schwankungen der Orientierungsbestimmung kann jedoch eine geringe verbleibende Winkelabweichung auftreten.

Insgesamt zeigen die Testfahrten, dass die entwickelte Endpunktkorrektur erfolgreich in die bestehende autonome Fahrzeugsteuerung integriert werden konnte. Sowohl die translatorische Korrektur als auch die anschließende Anpassung der Endausrichtung wurden bei allen untersuchten Fahrten ausgeführt. Die Versuche bestätigen damit die grundsätzliche Funktionsfähigkeit des entwickelten Verfahrens. Für eine quantitative Bewertung der erreichbaren Positionsgenauigkeit wäre in zukünftigen Versuchen jedoch eine unabhängige Referenzmessung der tatsächlichen Fahrzeugpose erforderlich.

Kapitel 4

Zusammenfassung und Ausblick

4.1 Zusammenfassung

Im Rahmen dieser Projektarbeit wurde die bestehende autonome Fahrzeugsteuerung des Nimbulus-E um eine kartenbasierte Live-Lokalisierung und eine Endpunktkorrektur erweitert. Ziel war es, die Fahrzeugpose während der autonomen Fahrt innerhalb einer zuvor aufgenommenen Umgebungskarte zu schätzen und nach Abschluss einer Trajektorie verbleibende Abweichungen von der geplanten Endpose zu korrigieren. Die bereits vorhandene Fahrzeugsteuerung, die odometrische Positionsbestimmung und die Lidar-Sensorik bildeten hierfür die Grundlage.

Für die Live-Lokalisierung wurde eine Monte-Carlo-Lokalisation implementiert, welche die odometrisch bestimmte Fahrzeugbewegung mit aktuellen Lidar-Scans und einer bekannten Belegungskarte kombiniert. Anhand mehrerer Testfahrten wurden insbesondere die Partikelanzahl und die Aktualisierungsschwellwerte untersucht. Die MCL war grundsätzlich in der Lage, den Bewegungsverlauf des Fahrzeugs innerhalb der Karte abzubilden. Gleichzeitig zeigte sich ein Zielkonflikt zwischen der Stabilität der Positionsschätzung und dem erforderlichen Rechenaufwand. Eine maximale Partikelanzahl von 5000 stellte unter den untersuchten Konfigurationen den geeignetsten Kompromiss dar. Die resultierende Positionsschätzung wies jedoch weiterhin Schwankungen auf. Da zudem keine unabhängige Referenzmessung der tatsächlichen Fahrzeugpose zur Verfügung stand, konnte die Genauigkeit der MCL gegenüber der Odometrie nicht quantitativ bestimmt werden. Für die Endpunktkorrektur wurde die MCL-Schätzung daher in der vorliegenden Implementierung nicht verwendet.

Die Endpunktkorrektur wurde stattdessen auf Grundlage der odometrisch bestimmten Fahrzeugpose umgesetzt. Nach Abschluss der ursprünglichen Trajektorie wird zunächst die Abweichung von der geplanten Endposition bestimmt und bei Bedarf eine geradlinige translatorische Korrekturbewegung erzeugt. Anschließend wird die Orientierung des Fahrzeugs durch eine Rotation um den Fahrzeugmittelpunkt angepasst. Die Funktionsfähigkeit des Verfahrens wurde anhand von zwei Kreisfahrten und zwei Fahrten entlang einer Valley-Trajektorie untersucht. Bei allen vier Versuchen lag die odometrisch bestimmte Positionsabweichung vor Beginn der Korrektur oberhalb der festgelegten Toleranz, sodass eine Korrekturbewegung ausgeführt wurde. Die Versuche

zeigten qualitativ, dass sich das Fahrzeug dabei in Richtung des vorgesehenen Endpunktes bewegte und auch die anschließende Orientierungskorrektur ausgeführt werden konnte. Eine quantitative Bestimmung des tatsächlich verbleibenden Positionsfehlers war aufgrund der fehlenden unabhängigen Referenzmessung jedoch nicht möglich.

Insgesamt konnten mit der Monte-Carlo-Lokalisation und der Endpunktkorrektur beide im Rahmen der Projektarbeit untersuchten Erweiterungen in die bestehende autonome Fahrzeugsteuerung integriert und unter realen Fahrbedingungen erprobt werden. Die Ergebnisse zeigen zugleich die derzeitigen Grenzen des Systems. Insbesondere eine quantitative Bewertung und weitere Verbesserung der Lokalisierung sowie eine präzisere Rückführung der tatsächlichen Fahrzeugpose während der Korrektur bieten Ansatzpunkte für die weitere Entwicklung des Nimbulus-E.

4.2 Ausblick

Die im Rahmen dieser Arbeit entwickelten Erweiterungen bieten mehrere Ansatzpunkte für die weitere Verbesserung der autonomen Navigation des Nimbulus-E. Ein wesentlicher nächster Schritt besteht in der Weiterentwicklung der kartenbasierten Live-Lokalisierung. Insbesondere könnten die Parametrisierung der Monte-Carlo-Lokalisation und die Verarbeitung der Lidar-Daten weiter optimiert werden. Darüber hinaus wäre eine systematische Untersuchung der benötigten Berechnungszeiten sinnvoll. In den durchgeführten Versuchen traten bei einer Erhöhung der maximalen Partikelanzahl Probleme bei der autonomen Trajektorienausführung auf. Ob diese tatsächlich auf den erhöhten Rechenaufwand der Lokalisierung zurückzuführen sind, konnte im Rahmen dieser Arbeit jedoch nicht abschließend bestimmt werden. Aufbauend auf entsprechenden Messungen könnte untersucht werden, ob eine Optimierung der Implementierung oder eine leistungsfähigere Rechenplattform höhere Partikelzahlen beziehungsweise häufigere Aktualisierungen ermöglicht, ohne den zeitlichen Ablauf der Fahrzeugsteuerung zu beeinträchtigen.

Eine weiterentwickelte Live-Lokalisierung könnte zukünftig unmittelbar in die autonome Fahrzeugregelung einbezogen werden. Während die in dieser Arbeit entwickelte Korrektur erst nach Abschluss der ursprünglich geplanten Trajektorie eingreift, könnte die aktuelle Ist-Pose zukünftig bereits während der Fahrt kontinuierlich mit der vorgesehenen Bewegung verglichen werden. Bei auftretenden Abweichungen könnte das Fahrzeug zur geplanten Trajektorie zurückgeführt oder ausgehend von der aktuellen Pose eine neue Trajektorie zum Ziel berechnet werden. Auf diese Weise könnte die bisher nachgeschaltete Korrektur langfristig zu einer kontinuierlichen Online-Korrektur erweitert werden. Ein entsprechender Ansatz wurde bereits in der Vorarbeit als möglicher weiterer Entwicklungsschritt betrachtet [3].

Auch die entwickelte Endpunktkorrektur selbst kann weiter verbessert werden. Die translatorische Korrekturtrajektorie wird derzeit einmalig aus der nach Abschluss der ursprünglichen Fahrt bestimmten odometrischen Pose berechnet und anschließend ohne erneute Planung abgefahren. Zukünftig könnte die Fahrzeugpose auch während der

Korrekturbewegung fortlaufend berücksichtigt werden. Die verbleibende Abweichung zum Ziel könnte dadurch wiederholt bestimmt und die Korrekturbewegung entsprechend angepasst werden. Auch die anschließende Korrektur der Endausrichtung könnte durch eine präzisere Orientierungsbestimmung und eine feinere Regelung der Rotationsbewegung weiterentwickelt werden.

Für die Entwicklung und Bewertung solcher Verfahren wäre zudem eine unabhängige Referenzmessung der tatsächlichen Fahrzeugpose von besonderem Nutzen. In den durchgeführten Versuchen konnten die Odometrie und die Monte-Carlo-Lokalisation miteinander sowie mit der geplanten Trajektorie verglichen werden. Ohne eine unabhängige Bestimmung der realen Fahrzeugposition und -orientierung lässt sich daraus jedoch keine absolute Lokalisierungsgenauigkeit ableiten. Ein geeignetes externes Referenzsystem würde es ermöglichen, Positions- und Orientierungsfehler quantitativ zu bestimmen und unterschiedliche Lokalisierungs- und Korrekturverfahren objektiv miteinander zu vergleichen. Gleichzeitig könnte damit der tatsächlich verbleibende Fehler nach einer Endpunktkorrektur bestimmt werden.

Langfristig könnten die bisher weitgehend getrennten Komponenten für Kartierung, Trajektorienplanung, Lokalisierung und Korrektur zu einem geschlossenen Navigationssystem verbunden werden. Das Fahrzeug würde dann nicht ausschließlich eine vorab berechnete Trajektorie ausführen, sondern seine Pose während der Fahrt fortlaufend schätzen und seine Bewegung anhand der aktuellen Abweichung anpassen. Die in dieser Arbeit umgesetzte Live-Lokalisierung und Endpunktkorrektur stellen hierfür weitere Bausteine innerhalb der bestehenden autonomen Fahrzeugsteuerung des Nimbulus-E dar.

Anhang A

Code-Listings

In diesem Anhang sind die für die autonome Fahrzeugsteuerung und die im Rahmen des Projekts entwickelten Erweiterungen relevanten MATLAB-Dateien aufgeführt.

A.1 Trajektorien

A.1.1 Kreis-Trajektorie

Listing A.1: buildCircleTrajectory.m

```
1  function x = buildCircleTrajectory(R, tStep, v0)
2  %% INPUT:
3  % R: Radius in m
4  % tStep: time interval between two
5  %   discrete trajectory points in ms
6  % v0: initial/continous velocity in m/s
7  %
8  %% OUTPUT:
9  % x = [t, x, y, psi, xdot, ydot, psidot]
10 % in world coordinates and meters and radians and seconds
11
12 tStep = tStep / 1000; % for calculation in seconds
13 T = 2*pi*R/v0;
14 discretePoints = ceil(T/tStep);
15
16 x = zeros(discretePoints, 7);
17
18 x(1,1) = 0;
19 x(1,2) = 0;
20 x(1,3) = 0;
21 x(1,4) = pi/2;
22 x(1,5) = 0;
23 x(1,6) = v0;
24 x(1,7) = v0/R;
25
```

```
for i = 2:discretePoints+1
  x(i,1) = (i-1) * tStep;
  x(i,2) = R*cos((i-1)*2*pi/discretePoints)-R;
  x(i,3) = R*sin((i-1)*2*pi/discretePoints);
  x(i,4) = pi/2+(i-1)*2*pi/discretePoints;
  x(i,5) = (x(i,2) - x(i-1,2)) / tStep;
  x(i,6) = (x(i,3) - x(i-1,3)) / tStep;
  x(i,7) = (x(i,4) - x(i-1,4)) / tStep;
end

end
```

A.1.2 Gerade Trajektorie

Listing A.2: buildStraightTrajectory.m

```
function x = buildStraightTrajectory(tStep, distance)
%% INPUT:
% tStep: time interval between two discrete trajectory points [ms]
% distance: total distance to travel [m]
%
%% OUTPUT:
% x = [t, x, y, psi, xdot, ydot, psidot]
% world coordinates, meters, radians, seconds

tStep = tStep / 1000;   % convert to seconds

% Number of points (including start)
discretePoints = ceil(distance / tStep) + 1;

% Total time
T = (discretePoints - 1) * tStep;

% Constant velocity computed from distance and time
v = distance / T;

x = zeros(discretePoints, 7);

% Initial state
x(1,1) = 0;         % time
x(1,2) = 0;         % x-position
x(1,3) = 0;         % y-position
x(1,4) = pi/2;     % heading (phi)
x(1,5) = 0;         % xdot
x(1,6) = v;         % ydot
x(1,7) = 0;         % psidot

for i = 2:discretePoints
    x(i,1) = (i-1) * tStep;
```

```
34      x(i,2) = 0;
35      x(i,3) = v * x(i,1);
36      x(i,4) = pi/2;
37
38      x(i,5) = 0;
39      x(i,6) = v;
40      x(i,7) = 0;
41  end
42
43  end
```

A.1.3 Valley-Trajektorie

Listing A.3: buildValleyTrajectory.m

```
1  function traj = buildValleyTrajectory(tStep, vStraight, vCurve, R, pauseTime
       )
2  %BUILDVALLEYTRAJECTORY Creates the complete valley-shaped trajectory.
3  %
4  % INPUT:
5  %   tStep      : time step between trajectory points [ms]
6  %   vStraight  : translational velocity on straight segments [m/s]
7  %   vCurve     : translational velocity in curves [m/s]
8  %   R          : radius of the four 90-degree curves [m]
9  %   pauseTime  : pause after every straight and every curve [s]
10 %
11 % OUTPUT:
12 %   traj = [t, x, y, psi, xdot, ydot, psidot]
13 %
14 % Sign convention:
15 %   positive dPsi  -> left turn
16 %   negative dPsi  -> right turn
17 %
18 % Example:
19 %   tStep      = 20;
20 %   vStraight  = 0.10;
21 %   vCurve     = 0.04;
22 %   R          = 0.60;
23 %   pauseTime  = 0.80;
24 %
25 %   traj = buildValleyTrajectory( ...
26 %       tStep, vStraight, vCurve, R, pauseTime);
27
28     arguments
29         tStep      (1,1) double {mustBePositive}
30         vStraight  (1,1) double {mustBePositive}
31         vCurve     (1,1) double {mustBePositive}
32         R          (1,1) double {mustBePositive}
33         pauseTime  (1,1) double {mustBeNonnegative}
```

```
34  end
35
36  dt = tStep / 1000;
37
38  % State:
39  % [time, x, y, psi]
40  %
41  % psi = pi:
42  % Start on the right, pointing to the left.
43  state = [0, 0, 0, 0];
44
45  % Initial point: vehicle standing still.
46  traj = stateToTrajectoryRow(state, 0, 0);
47
48  sequence = {
49      'straight',  1
50      'turn',       pi/2
51      'straight',  0.5
52      'turn',      -pi/2
53      'straight',  0.5
54      'turn',      -pi/2
55      'straight',  0.5
56      'turn',       pi/2
57      'straight',  0.5
58  };
59
60  for k = 1:size(sequence,1)
61
62      segmentType = sequence{k,1};
63      value       = sequence{k,2};
64
65      switch segmentType
66
67          case 'straight'
68              seg = straightSegment( ...
69                  state, value, vStraight, dt);
70
71          case 'turn'
72              seg = circleSegment( ...
73                  state, R, value, vCurve, dt);
74
75          otherwise
76              error('Unknown segment type: %s', segmentType);
77      end
78
79      % Add movement segment.
80      traj = [traj; seg]; %#ok<AGROW>
81
```

```
82          % New state is the last pose of the segment.
83          state = traj(end,1:4);
84
85          % Add an explicit stop segment after every straight
86          % and every curve.
87          %
88          % The control loop can therefore continue sending
89          % zero velocities to the Arduino/VESC.
90          if pauseTime > 0
91
92              pauseSeg = pauseSegment( ...
93                  state, pauseTime, dt);
94
95              traj = [traj; pauseSeg]; %#ok<AGROW>
96
97              state = traj(end,1:4);
98          end
99      end
100 end
101
102
103 function row = stateToTrajectoryRow(state, v, omega)
104 %STATETOTRAJECTORYROW Converts a state into one trajectory row.
105
106     psi = state(4);
107
108     row = [
109         state(1), ...
110         state(2), ...
111         state(3), ...
112         psi, ...
113         v*cos(psi), ...
114         v*sin(psi), ...
115         omega
116     ];
117 end
118
119
120 function seg = straightSegment(state, distance, v, dt)
121 %STRAIGHTSEGMENT Creates one straight trajectory segment.
122
123     t0   = state(1);
124     x0   = state(2);
125     y0   = state(3);
126     psi0 = state(4);
127
128     % Required duration for the straight segment.
129     T = distance / v;
```

```
130
131     sampleTimes = makeSampleTimes(T, dt);
132     N = numel(sampleTimes);
133
134     seg = zeros(N,7);
135
136     for i = 1:N
137
138         tau = sampleTimes(i);
139
140         x = x0 + v*cos(psi0)*tau;
141         y = y0 + v*sin(psi0)*tau;
142
143         seg(i,:) = [
144             t0 + tau, ...
145             x, ...
146             y, ...
147             psi0, ...
148             v*cos(psi0), ...
149             v*sin(psi0), ...
150             0
151         ];
152     end
153
154     % Force the exact endpoint.
155     seg(end,2) = x0 + distance*cos(psi0);
156     seg(end,3) = y0 + distance*sin(psi0);
157 end
158
159
160 function seg = circleSegment(state, R, dPsi, v, dt)
161 %CIRCLESEGMENT Creates one circular trajectory segment.
162 %
163 % dPsi > 0: left turn
164 % dPsi < 0: right turn
165
166     t0   = state(1);
167     x0   = state(2);
168     y0   = state(3);
169     psi0 = state(4);
170
171     turnSign = sign(dPsi);
172
173     % Constant angular velocity in the curve.
174     omega = turnSign * v / R;
175
176     % Required duration for the angular change.
177     T = abs(dPsi) / abs(omega);
```

```
178
179     sampleTimes = makeSampleTimes(T, dt);
180     N = numel(sampleTimes);
181
182     seg = zeros(N,7);
183
184     % Circle centre.
185     cx = x0 - turnSign*R*sin(psi0);
186     cy = y0 + turnSign*R*cos(psi0);
187
188     for i = 1:N
189
190         tau = sampleTimes(i);
191         psi = psi0 + omega*tau;
192
193         x = cx + turnSign*R*sin(psi);
194         y = cy - turnSign*R*cos(psi);
195
196         seg(i,:) = [
197             t0 + tau, ...
198             x, ...
199             y, ...
200             psi, ...
201             v*cos(psi), ...
202             v*sin(psi), ...
203             omega
204         ];
205     end
206
207     % Force the exact endpoint and final orientation.
208     psiEnd = psi0 + dPsi;
209
210     seg(end,2) = cx + turnSign*R*sin(psiEnd);
211     seg(end,3) = cy - turnSign*R*cos(psiEnd);
212     seg(end,4) = psiEnd;
213     seg(end,5) = v*cos(psiEnd);
214     seg(end,6) = v*sin(psiEnd);
215     seg(end,7) = omega;
216 end
217
218
219 function seg = pauseSegment(state, pauseTime, dt)
220 %PAUSESEGMENT Keeps the vehicle at its current pose.
221 %
222 % During the pause:
223 %   xdot   = 0
224 %   ydot   = 0
225 %   psidot = 0
```

```
226 %
227 % Multiple trajectory samples are generated, so the controller can
228 % continue transmitting explicit stop commands.
229
230     t0   = state(1);
231     x0   = state(2);
232     y0   = state(3);
233     psi0 = state(4);
234
235     sampleTimes = makeSampleTimes(pauseTime, dt);
236     N = numel(sampleTimes);
237
238     seg = zeros(N,7);
239
240     for i = 1:N
241
242         seg(i,:) = [
243             t0 + sampleTimes(i), ...
244             x0, ...
245             y0, ...
246             psi0, ...
247             0, ...
248             0, ...
249             0
250         ];
251     end
252 end
253
254
255 function sampleTimes = makeSampleTimes(T, dt)
256 %MAKESAMPLETIMES Creates sample times without exceeding T.
257
258     if T <= 0
259         sampleTimes = [];
260         return;
261     end
262
263     sampleTimes = dt:dt:T;
264
265     % Add the exact endpoint if T is not a multiple of dt.
266     if isempty(sampleTimes) || ...
267             abs(sampleTimes(end)-T) > 10*eps(T)
268
269         sampleTimes = [sampleTimes, T];
270     end
271 end
```

A.2 Endpunktkorrektur

A.2.1 Positionskorrektur

Listing A.4: buildEndPointCorrectionTrajectory.m

```
function correctionTrajectory = buildEndPointCorrectionTrajectory( ...
    actualPose, targetPose, tStep, vCorrection)

%% buildEndPointCorrectionTrajectory
% Builds a straight correction trajectory from actualPose to targetPose.
%
% INPUT:
% actualPose  = [x, y, psi] current vehicle pose
% targetPose  = [x, y, psi] desired end pose
% tStep       = time step in ms
% vCorrection = correction velocity in m/s
%
% IMPORTANT:
% psi must be in radians.
%
% OUTPUT:
% correctionTrajectory = [t, x, y, psi, xdot, ydot, psidot]

  tStepSec = tStep / 1000;

  x0 = actualPose(1);
  y0 = actualPose(2);

  xEnd = targetPose(1);
  yEnd = targetPose(2);

  dxTotal = xEnd - x0;
  dyTotal = yEnd - y0;

  distance = sqrt(dxTotal^2 + dyTotal^2);

  % If vehicle is already close enough, no correction is needed
  tolerance = 0.05; % 5 cm
  if distance < tolerance
    correctionTrajectory = [];
    return;
  end

  % Heading of the straight correction line
  psiCorrection = actualPose(3);
  travelDirection = atan2(dyTotal,dxTotal);

```

```
43
44    % Number of discrete trajectory points
45    T = distance / vCorrection;
46    discretePoints = max(2, ceil(T / tStepSec) + 1);
47
48    correctionTrajectory = zeros(discretePoints, 7);
49
50    for i = 1:discretePoints
51      s = (i-1) / (discretePoints-1);
52
53      correctionTrajectory(i,1) = (i-1) * tStepSec;
54      correctionTrajectory(i,2) = x0 + s * dxTotal;
55      correctionTrajectory(i,3) = y0 + s * dyTotal;
56      correctionTrajectory(i,4) = psiCorrection;
57
58      correctionTrajectory(i,5) = vCorrection * cos(travelDirection);
59      correctionTrajectory(i,6) = vCorrection * sin(travelDirection);
60      correctionTrajectory(i,7) = 0;
61    end
62
63  end
```

A.2.2 Korrektur der Endausrichtung

Listing A.5: buildEndPointRotating2.m

```
1  function x = buildEndPointRotating2( ...
2      actualRot,endRot,tStep)
3  %% INPUT:
4  % T: period time for one single circle
5  % maxAngle: max angle to one side in degrees
6  % nofStages: number of full stages/cirlces (2*maxAngle)
7  %   car should rotate
8  % tStep: time interval between
9  %   two discrete trajectory points in ms
10 %
11 %% OUTPUT:
12 % x = [t, x, y, psi, xdot, ydot, psidot]
13 % in world coordinates and meters and radians and seconds
14 T= 2;
15 tStep = tStep/1000;
16 t = 0:tStep:T;
17 %angleVec = linspace(actualRot, endRot, length(t));
18 %maxAngleRad = maxAngleDeg/180*pi;
19 %psidotMax = 2 * pi/T * maxAngleRad;
20
21 deltaRot = atan2(sin(endRot - actualRot), cos(endRot - actualRot));
22 if(actualRot - endRot) < 0
23     psidot = 0.6;
```

```
24 else
25     psidot = -0.6;
26 end
27
28 if abs(actualRot-endRot) > pi
29     psidot = -psidot;
30 end
31
32 x = zeros(length(t),7);
33
34 % x(1,1) = 0;
35 % x(1,2) = 0;
36 % x(1,3) = 0;
37 % x(1,4) = actualRot;
38 % x(1,5) = 0;
39 % x(1,6) = 0;
40 % x(1,7) = psidotMax;
41
42 for i = 1:length(t)
43   s = (i-1)/(length(t)-1);
44
45   x(i,1) = t(i);
46   x(i,2) = 0;
47   x(i,3) = 0;
48   x(i,4) = actualRot + s * deltaRot; %maxAngleRad*sin(angleVec(i));
49   x(i,5) = 0;
50   x(i,6) = 0;
51   x(i,7) = psidot; %psidotMax * cos(angleVec(i));
52 end
53
54 end
```

A.3 Autonome Fahrzeugsteuerung

A.3.1 Umrechnung der Trajektorie

Listing A.6: trajectoryTo8DOF.m

```
1 function res = ...
2     trajectoryTo8DOF(psi, xdot_s, ydot_s, psidot, deltaLast)
3
4 % input in world coordinates and output in vehicle coordinates
5 % psi_input in rad
6 %
7 % deltaLast: last 4 steering angles [VL VR HL HR] in degree
8 %
9 % output
10 % res = [delta_fl omega_fl;
```

```
11 %        delta_fr omega_fr;
12 %        delta_rl omega_rl;
13 %        delta_rr omega_rr]
14 % unit: [deg      rounds/s]
15
16 % ------------------------------------------------------------
17 % vectors that point into the direction in which the wheel is
18 % steered in vehicle coordinates
19 % deltaVecs = [deltaVLxV, deltaVRxV, deltaHLxV, deltaHRxV;
20 %              deltaVLyV, deltaVRyV, deltaHLyV, deltaHRyV]
21
22 % x = [xdot_fl xdot_fr xdot_rl xdot_rr;
23 %      ydot_fl ydot_fr yfot_rl ydot_rr]
24 % unit: [m/s  m/s]
25
26
27 % predefinition of car params
28 track_width = 0.55;   % Spurweite in m
29 wheel_base = 0.86;    % Radstand in m
30 wheelRadius = 0.095;  % Radradius in m
31
32 % build rotational matrix
33 rotMat_V_in_W = [cos(psi) -sin(psi) 0;
34                  sin(psi)  cos(psi) 0;
35                  0                  0 1];
36 % vWorld = [vFl_W vFr_W vRl_W vRr_W]
37 % vVehicle = [vFl_V vFr_V vRl_V vRr_V]
38 vWorld_s(:,1) = [xdot_s; ydot_s; 0];
39 vVehicle_s(:,1) = rotMat_V_in_W' * vWorld_s(:,1);
40
41 % convert lastdeltas to rad
42 deltaLast = deltaLast * pi/180;
43
44 %
45 x(1,1) = vVehicle_s(1) - track_width/2 * psidot;
46 x(2,1) = vVehicle_s(2) + wheel_base/2 * psidot;
47 res(1,1) = findNearestTan(x(1,1),x(2,1), deltaLast(1));
48 deltaVecs(1:2,1) = [cos(res(1,1)), sin(res(1,1))];
49
50 x(1,2) = vVehicle_s(1) + track_width/2 * psidot;
51 x(2,2) = vVehicle_s(2) + wheel_base/2 * psidot;
52 res(2,1) = findNearestTan(x(1,2),x(2,2), deltaLast(2));
53 deltaVecs(1:2,2) = [cos(res(2,1)), sin(res(2,1))];
54
55 x(1,3) = vVehicle_s(1) - track_width/2 * psidot;
56 x(2,3) = vVehicle_s(2) - wheel_base/2 * psidot;
57 res(3,1) = findNearestTan(x(1,3),x(2,3), deltaLast(3));
58 deltaVecs(1:2,3) = [cos(res(3,1)), sin(res(3,1))];
```

```
59
60 x(1,4) = vVehicle_s(1) + track_width/2 * psidot;
61 x(2,4) = vVehicle_s(2) - wheel_base/2 * psidot;
62 res(4,1) = findNearestTan(x(1,4),x(2,4), deltaLast(4));
63 deltaVecs(1:2,4) = [cos(res(4,1)), sin(res(4,1))];
64
65 % convert angles from rad to degree
66 res(:, 1) = res(:, 1) .* 180/pi;
67 res(:, 1) = mod(res(:,1), 360);
68
69 % sign and magnitude of rotational rate
70 res(1,2) = sign(deltaVecs(:,1)'*[x(1,1); x(2,1)]) * ...
71   sqrt(x(1,1)^2+x(2,1)^2) / (wheelRadius * 2 * pi);
72 res(2,2) = sign(deltaVecs(:,2)'*[x(1,2); x(2,2)]) * ...
73   sqrt(x(1,2)^2+x(2,2)^2) / (wheelRadius * 2 * pi);
74 res(3,2) = sign(deltaVecs(:,3)'*[x(1,3); x(2,3)]) * ...
75   sqrt(x(1,3)^2+x(2,3)^2) / (wheelRadius * 2 * pi);
76 res(4,2) = sign(deltaVecs(:,4)'*[x(1,4); x(2,4)]) * ...
77   sqrt(x(1,4)^2+x(2,4)^2) / (wheelRadius * 2 * pi);
78
79 % for loop only for making all steering angles possible
80 % not necessary in unconstrained environment
81 for i = 1:4
82   if res(i, 1) >= 90 && res(i, 1) <= 180
83     res(i,1) = interp1([90,180],[-90 0], res(i,1));
84     res(i,2) = res(i,2) * (-1);
85   end
86   if res(i, 1) >= 180 && res(i, 1) <= 270
87     res(i,1) = interp1([180,270],[90 0], res(i,1));
88     res(i,2) = res(i,2) * (-1);
89   end
90   if res(i, 1) >= 270 && res(i, 1) <= 360
91     res(i,1) = interp1([270, 360],[-90 0], res(i,1));
92   end
93 end
94
95 end
```

A.3.2 Autonome Steuerung

Listing A.7: matlab_autonomous.m

```
1 clear
2 close all
3 clc
4 format short
5
6 warning('off', 'all'); % Disable warnings (use with caution)
7 % Disable lidar warnings at every scan
```

```
8
9  % CHANGE: save 'carPoses', 'odometries' and 'trajectoryFig'?
10 saveData = false;
11
12 % CHANGE: lidar false for simple driving, true for mapping
13 useLidar = true;
14
15 % CHANGE: choose trajectory to follow
16 %   "circle", "rot" and "rotAndMove" are precomputed test
17 %   trajectories. If non of these, trajectoryName must be a
18 %   valid filename of a .mat object containing an occupancy
19 %   map object called "occMap". According parameters for each
20 %   trajectory can be set further below
21 trajectoryName = "circle";
22 %trajectoryName = "rot";
23 %trajectoryName = "rot2EndRot";
24 %trajectoryName = "rotAndMove";
25 %trajectoryName = "projektTrajectory";
26 %trajectoryName = "occMap_test.mat";
27 %trajectoryName = "occMap1.mat";
28 %trajectoryName = "zero";
29 %trajectoryName = "valley";
30
31 mapFile = "recordedMap.mat";
32 defaultSize = 1000;
33 estimatePoses = zeros(defaultSize,4);
34 slamInterval = 5; % Time between SLAM updates in seconds
35
36 %% Arduino and controller setup
37 disp("Serialport Connection procedure started")
38 % CHANGE: set COM and baud rate
39 arduMega = serialport("COM4", 115200);
40 tic;
41 f = waitbar(0, "Waiting to set serial connection to Arduino");
42 % CHANGE: if connection to ardu fails, increase value
43 initTime = 10;
44 while toc < initTime
45   waitbar(toc/initTime, f, ...
46     "Waiting to setup serial connection to Arduino");
47 end
48 delete(f) % Close the waitbar
49 disp("Serialport Connection procedure finished")
50
51
52 %% Variables for Lidar and SLAM
53 if useLidar
54
55
```

```
56      MaxLidarRange = 10;
57      lastPose = [0 0 0];
58
59
60      load(mapFile,"occMap");
61      mcl = monteCarloLocalization;
62      mcl.UseLidarScan = true;
63      mcl.ParticleLimits = [500 5000];
64      mcl.UpdateThresholds = [0.05 0.05 0.01];
65
66      sensorModel = likelihoodFieldSensorModel;
67      sensorModel.Map = occMap;
68      sensorModel.SensorLimits = [0.2 MaxLidarRange];
69      sensorModel.SensorPose = [0 0 0];
70      mcl.SensorModel = sensorModel;
71      mcl.InitialPose = [0 0 0];
72      mcl.InitialCovariance = diag([0.05 0.05 0.1]);
73
74
75
76
77    % Setup and start Velodyne Lidar
78    lidar = velodynelidar('VLP32C');
79    start(lidar);
80    while toc < slamInterval % ensure one lidar scan at start
81      waitbar(toc/slamInterval, f, 'Waiting to setup lidar');
82    end
83    delete(f)
84    disp("Lidar and parallel pool ready");
85  end
86
87  %% calculate trajectory
88  disp("calc trajectory for " + trajectoryName)
89
90  % CHANGE: adjust parameters for chosen trajectory
91  tStep = 100; % in ms ATTENTION must be same as on Arduino
92  switch trajectoryName
93
94    case "circle"
95      R = 0.8;    % radius in meters
96      v0 = 0.4; % speed in m/s
97
98      trajectory = buildCircleTrajectory(R, tStep, v0);
99    case "rot"
100     pendulumT = 3;    % period time of rotation in s
101     maxAngleDeg = 45; % angle to one side in degree
102     nofStages = 2;    % how many complete cycles
103
```

```
104     trajectory = buildRotatingTrajectory( ...
105       pendulumT, maxAngleDeg, nofStages, tStep);
106   case "rot2EndRot"
107     actualRot = 0;
108     endRot = pi/2;
109     maxAngleDeg = 45; % angle to one side in degree
110
111     %trajectory = buildEndPointRotating(actualRot,endRot,maxAngleDeg,tStep);
112     trajectory = buildEndPointRotating2(actualRot,endRot,tStep);
113
114   case "valley"
115     v_straight = 0.8;
116     v_curve = 0.5;
117     R = 0.8;
118     pausetime = 1;
119     trajectory = buildValleyTrajectory(tStep, v_straight,v_curve, R,
      pausetime);
120
121   case "projektTrajectory"
122         v = 0.5; % constant velocity [m/s]
123         s = 1; % first straight distance [m]
124         R = 1; % radius for all circles [m]
125         d2 = 1; % middle straight distance [m]
126         d = 1; % final straight distance [m]
127         a = 0.5; % acceleration in [m/s^2]
128         trajectory = buildTrajectorywithAcceleration(tStep, v, s, R, d2, d,a
      );
129   case "rotAndMove"
130     pendulumT = 3;     % period time of rotation in s
131     maxAngleDeg = 45; % angle to one side in degree
132     nofStages = 2;     % how many complete cycles
133     distance = 10;     % distance forward in meters
134
135     trajectory = buildRotatingAndMovingTrajectory( ...
136       pendulumT, maxAngleDeg, distance, nofStages, tStep);
137 case "zero"
138     totalTime = 5; % End Time
139     dt = tStep / 1000;   % convert to seconds
140     N  = ceil(totalTime / dt);
141
142     % Initial pose (same as your trajectory generator)
143     x0 = 0;
144     y0 = 0;
145     psi0 = pi/2;
146
147     trajectory = zeros(N,7);
148
149     for i = 1:N
```

```
150         t = i*dt;
151
152         trajectory(i,:) = [t, ...
153                     x0, ...
154                     y0, ...
155                     psi0, ...
156                     0, ...   % xdot
157                     0, ...   % ydot
158                     0];      % psidot
159     end
160   otherwise
161     % safeguard against non existing files
162     if ~exist(trajectoryName, "file")
163       error("filename " + trajectoryName + " does not exist!")
164     end
165     variableInfo = who('-file', trajectoryName);
166     if ~ismember('occMap', variableInfo)
167       error(trajectoryName + " does not contain an" + ...
168         " occupancy  map called 'occMap'!")
169     end
170
171     vMax = 2;               % speed in m/s
172     x0 = [6 13 0];          % in [m m rad]
173     xEnd = [14 11 -pi/2]; % in [m m rad]
174
175     [trajectory, trajectoryFig] = pathPlanningHybridAStar(trajectoryName,
      tStep, vMax, x0, xEnd);
176 end
177 disp("trajectory calculation finished")
178
179 sv = zeros(4, 2, length(trajectory));
180 deltaLast = zeros(4,1);
181
182 disp("convert to 8 values for car")
183 for i = 1:length(trajectory)
184   sv(:, :, i) = trajectoryTo8DOF( ...
185     trajectory(i,4), trajectory(i,5), trajectory(i,6), ...
186     trajectory(i,7), deltaLast);
187   deltaLast = sv(:, 1, i);
188 end
189
190 %%
191 disp("send to initial steering angles")
192 str = sprintf("<%.0f, %.0f, %.0f, %.0f, 0, 0, 0, 0>", ...
193   sv(1,1,1), sv(2,1,1), sv(3,1,1), sv(4,1,1));
194 write(arduMega, str, "string");
195 pause(5);
196
```

```
197 %% preallocation
198
199 currentSteeringAngles = zeros(4,1);
200 currentRotRates = zeros(4,1);
201 carPoses = zeros(length(trajectory), 4);
202 carPoses(1,:) = trajectory(1,1:4);
203 carPoses(1,4) = rad2deg(carPoses(1,4));
204 lastPsiInDeg = 0;
205 lastTimestamp = 0;
206 odometries = zeros(2,4,length(trajectory));
207 estimatedPoses = zeros(defaultSize, 3);
208
209 %% communication with Arduino
210 disp("start trajectory following")
211 i = 2;
212 flush(arduMega, "input");
213 tic
214 while i <= length(trajectory)
215
216   % send values when asked for
217   % read serial data from arduino if available
218   if arduMega.NumBytesAvailable > 0
219     % send next trajectory value
220     str = sprintf("<%.0f, %.0f, %.0f, %.0f, " + ...
221                   "%.2f, %.2f, %.2f, %.2f>", ...
222       sv(1,1,i), sv(2,1,i), sv(3,1,i), sv(4,1,i), ...
223       sv(1,2,i), sv(2,2,i), sv(3,2,i), sv(4,2,i));
224
225     write(arduMega, str, "string");
226     toc
227
228     data = readline(arduMega);    % read data from arduino
229     data = extractBetween(data, '<', '>');
230     dataComponents = split(data, ',');
231
232     % save incoming data
233     timestamp = str2double(dataComponents{1});
234     psiInDeg = mod(str2double(dataComponents{2}), 360);
235     currentSteeringAngles(1) = str2double(dataComponents{3});
236     currentSteeringAngles(2) = str2double(dataComponents{4});
237     currentSteeringAngles(3) = str2double(dataComponents{5});
238     currentSteeringAngles(4) = str2double(dataComponents{6});
239     currentRotRates(1) = str2double(dataComponents{7});
240     currentRotRates(2) = str2double(dataComponents{8});
241     currentRotRates(3) = str2double(dataComponents{9});
242     currentRotRates(4) = str2double(dataComponents{10});
243
244     odometries(:, :, i) = ...
```

```
245       [currentSteeringAngles'; currentRotRates'];
246
247     deltaTinS = (timestamp - lastTimestamp) / 1000;
248     deltaPsi = psiInDeg - lastPsiInDeg;
249     if deltaPsi > 180
250       deltaPsi = deltaPsi - 360;
251     elseif deltaPsi < -180
252       deltaPsi = deltaPsi + 360;
253     end
254     psidotInDeg = deltaPsi / deltaTinS;
255     lastTimestamp = timestamp;
256     lastPsiInDeg = psiInDeg;
257
258     [dx, dy] = odometry8DOFtoRelPose( ...
259       currentSteeringAngles, currentRotRates, ...
260       psiInDeg, psidotInDeg, deltaTinS);
261
262     carPoses(i,1) = timestamp;
263     carPoses(i,2) = carPoses(i-1,2) + dx;
264     carPoses(i,3) = carPoses(i-1,3) + dy;
265     carPoses(i,4) = psiInDeg;
266
267 if useLidar
268   [frame, ~] = read(lidar, 'latest');
269   scan = pc2scan(frame);
270
271   odomPose = carPoses(i, [2,3,4]);
272   odomPose(3) = deg2rad(odomPose(3));
273
274   [isUpdated, estimatedPose, covariance] = mcl(odomPose,scan);
275
276    estimatedPoses(i,:) = estimatedPose;
277 end
278
279     i = i + 1;
280   end
281 end
282
283 %% End point correction
284 write(arduMega, "<0,0,0,0, 0,0,0,0>", "string");
285
286 poseIdx = i-1;
287 plannedEndPose = trajectory(end, 2:4);
288
289 actualEndPose = carPoses(poseIdx, 2:4);
290 actualEndPose(3) = deg2rad(actualEndPose(3));
291
292 correctionTrajectory = buildEndPointCorrectionTrajectory( ...
```

```
293     actualEndPose, plannedEndPose, tStep, 0.5);
294
295 if ~isempty(correctionTrajectory)
296
297     disp("Starting end point correction...");
298
299     correctionControlValues = zeros(size(correctionTrajectory,1), 8);
300
301     deltaLast = zeros(1,4);
302
303     for k = 1:size(correctionTrajectory,1)
304
305         res = trajectoryTo8DOF( ...
306             correctionTrajectory(k,4), ...
307             correctionTrajectory(k,5), ...
308             correctionTrajectory(k,6), ...
309             correctionTrajectory(k,7), ...
310             deltaLast);
311
312         correctionControlValues(k,:) = [res(:,1)' res(:,2)'];
313         deltaLast = res(:,1)';
314
315     end
316 pause(1);
317  sv = correctionControlValues(1,:);
318  disp("send to initial steering angles")
319 str = sprintf("<%.0f, %.0f, %.0f, %.0f, 0, 0, 0, 0>", ...
320   sv(1), sv(2), sv(3), sv(4));
321 write(arduMega, str, "string");
322 pause(1);
323
324     for k = 1:size(correctionControlValues,1)
325     if arduMega.NumBytesAvailable > 0
326         sv = correctionControlValues(k,:);
327
328         str = sprintf( ...
329             "<%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f>", ...
330             sv(1), sv(2), sv(3), sv(4), ...
331             sv(5), sv(6), sv(7), sv(8));
332
333     write(arduMega, str, "string");
334     data = readline(arduMega);    % read data from arduino
335     data = extractBetween(data, '<', '>');
336     dataComponents = split(data, ',');
337
338     % save incoming data
339     timestamp = str2double(dataComponents{1});
340     psiInDeg = mod(str2double(dataComponents{2}), 360);
```

```
341     currentSteeringAngles(1) = str2double(dataComponents{3});
342     currentSteeringAngles(2) = str2double(dataComponents{4});
343     currentSteeringAngles(3) = str2double(dataComponents{5});
344     currentSteeringAngles(4) = str2double(dataComponents{6});
345     currentRotRates(1) = str2double(dataComponents{7});
346     currentRotRates(2) = str2double(dataComponents{8});
347     currentRotRates(3) = str2double(dataComponents{9});
348     currentRotRates(4) = str2double(dataComponents{10});
349
350     odometries(:, :, poseIdx) = ...
351       [currentSteeringAngles'; currentRotRates'];
352
353     deltaTinS = (timestamp - lastTimestamp) / 1000;
354     deltaPsi = psiInDeg - lastPsiInDeg;
355     if deltaPsi > 180
356       deltaPsi = deltaPsi - 360;
357     elseif deltaPsi < -180
358       deltaPsi = deltaPsi + 360;
359     end
360     psidotInDeg = deltaPsi / deltaTinS;
361     lastTimestamp = timestamp;
362     lastPsiInDeg = psiInDeg;
363
364     [dx, dy] = odometry8DOFtoRelPose( ...
365       currentSteeringAngles, currentRotRates, ...
366       psiInDeg, psidotInDeg, deltaTinS);
367     poseIdx = poseIdx+1;
368     carPoses(poseIdx,1) = timestamp;
369     carPoses(poseIdx,2) = carPoses(poseIdx-1,2) + dx;
370     carPoses(poseIdx,3) = carPoses(poseIdx-1,3) + dy;
371     carPoses(poseIdx,4) = psiInDeg;
372     end
373     if useLidar
374   [frame, ~] = read(lidar, 'latest');
375   scan = pc2scan(frame);
376
377   odomPose = carPoses(poseIdx, [2,3,4]);
378   odomPose(3) = deg2rad(odomPose(3));
379
380   [isUpdated, estimatedPose, covariance] = mcl(odomPose,scan);
381
382    estimatedPoses(poseIdx,:) = estimatedPose;
383     end
384
385     end
386
387     disp("End point correction finished.");
388
```

```
389 end
390 %% Angle correction
391     actualRot = deg2rad(carPoses(poseIdx,4));
392     actualRot_deg = rad2deg(actualRot);
393     endRot = actualEndPose(3);
394     endRot_rad = 2*pi;
395     endRot_deg = rad2deg(endRot_rad);
396     maxAngleDeg = 45; % angle to one side in degree
397
398     %trajectory = buildEndPointRotating(actualRot,endRot,maxAngleDeg,tStep);
399     trajectory_EndPointRotating = buildEndPointRotating2(actualRot,
        endRot_rad,tStep);
400     sv = zeros(4, 2, length(trajectory_EndPointRotating));
401 deltaLast = zeros(4,1);
402
403 disp("convert to 8 values for car")
404 for i = 1:length(trajectory_EndPointRotating)
405   sv(:, :, i) = trajectoryTo8DOF( ...
406     trajectory_EndPointRotating(i,4), trajectory_EndPointRotating(i,5),
       trajectory_EndPointRotating(i,6), ...
407     trajectory_EndPointRotating(i,7), deltaLast);
408   deltaLast = sv(:, 1, i);
409 end
410
411 disp("send to initial steering angles")
412 str = sprintf("<%.0f, %.0f, %.0f, %.0f, 0, 0, 0, 0>", ...
413   sv(1,1,1), sv(2,1,1), sv(3,1,1), sv(4,1,1));
414 write(arduMega, str, "string");
415 pause(1);
416
417 %% communication with Arduino
418 disp("start trajectory following")
419 n = 1;
420 flush(arduMega, "input");
421 delta_deg = 5;
422 delta_rad = deg2rad(delta_deg);
423 psiInRad = deg2rad(carPoses(end,4));
424
425 if endRot_deg < 0
426     endRot_deg = 360 + endRot_deg;
427 end
428
429 while abs((carPoses(poseIdx-1,4)) - endRot_deg) > delta_deg && n < 100
430   % send values when asked for
431   % read serial data from arduino if available
432   if arduMega.NumBytesAvailable > 0
433     % send next trajectory value
434     str = sprintf("<%.0f, %.0f, %.0f, %.0f, " + ...
```

```
435                     "%.2f, %.2f, %.2f, %.2f>", ...
436       sv(1,1,1), sv(2,1,1), sv(3,1,1), sv(4,1,1), ...
437       sv(1,2,1), sv(2,2,1), sv(3,2,1), sv(4,2,1));
438     write(arduMega, str, "string");
439
440     data = readline(arduMega);    % read data from arduino
441     data = extractBetween(data, '<', '>');
442     dataComponents = split(data, ',');
443
444     % save incoming data
445     timestamp = str2double(dataComponents{1});
446     psiInRad = deg2rad(str2double(dataComponents{2}));
447     psiInDeg = mod(str2double(dataComponents{2}), 360);
448     currentSteeringAngles(1) = str2double(dataComponents{3});
449     currentSteeringAngles(2) = str2double(dataComponents{4});
450     currentSteeringAngles(3) = str2double(dataComponents{5});
451     currentSteeringAngles(4) = str2double(dataComponents{6});
452     currentRotRates(1) = str2double(dataComponents{7});
453     currentRotRates(2) = str2double(dataComponents{8});
454     currentRotRates(3) = str2double(dataComponents{9});
455     currentRotRates(4) = str2double(dataComponents{10});
456 odometries(:, :, poseIdx) = ...
457       [currentSteeringAngles'; currentRotRates'];
458
459     deltaTinS = (timestamp - lastTimestamp) / 1000;
460     deltaPsi = psiInDeg - lastPsiInDeg;
461     if deltaPsi > 180
462       deltaPsi = deltaPsi - 360;
463     elseif deltaPsi < -180
464       deltaPsi = deltaPsi + 360;
465     end
466     psidotInDeg = deltaPsi / deltaTinS;
467     lastTimestamp = timestamp;
468     lastPsiInDeg = psiInDeg;
469
470     [dx, dy] = odometry8DOFtoRelPose( ...
471       currentSteeringAngles, currentRotRates, ...
472       psiInDeg, psidotInDeg, deltaTinS);
473     poseIdx = poseIdx+1;
474     carPoses(poseIdx,1) = timestamp;
475     carPoses(poseIdx,2) = carPoses(poseIdx-1,2) + dx;
476     carPoses(poseIdx,3) = carPoses(poseIdx-1,3) + dy;
477     carPoses(poseIdx,4) = psiInDeg;
478     pause(tStep / 1000);
479
480 if useLidar
481   [frame, ~] = read(lidar, 'latest');
482   scan = pc2scan(frame);
```

```
483
484   odomPose = carPoses(poseIdx, [2,3,4]);
485   odomPose(3) = deg2rad(odomPose(3));
486
487   [isUpdated, estimatedPose, covariance] = mcl(odomPose,scan);
488
489    estimatedPoses(poseIdx,:) = estimatedPose;
490   end
491     n = n + 1;
492   end
493 end
494
495 %% leave program clean
496 write(arduMega, "<0,0,0,0, 0,0,0,0>", "string");
497 pause(2);
498
499 delete(arduMega)
500 disp("Serial Port Object Arduino Mega cleared!")
501 % Stop Lidar and close SLAM processing
502 if useLidar
503   disp('Stopping Lidar and SLAM processing.');
504   stop(lidar); % Stop the Lidar sensor
505   clear lidar; % Release resources associated with Lidar
506
507 end
508 %%  Plot
509 % remove preallocated memory
510 %dataTable(i:end, :) = [];
511 %CarPoses(i:end, :) = [];
512 estimatedPoses(poseIdx:end, :) = [];
513 % set first angle as zero position
514 % carPoses(:,1) = carPoses(:,1) - carPoses(1,1);
515
516 %% plot calculated world poses
517 figure;
518 axis("equal");
519 for n = 1:poseIdx-1
520   plot(carPoses(n,2),carPoses(n,3), 'ro');
521   drawnow;
522   hold on;
523 end
524 xlabel('x', Interpreter='latex');
525 ylabel('y', Interpreter='latex');
526 hold off;
527
528 %% plot slam results
529 if useLidar
530
```

```
figure;
axis("equal");
for n = 1:poseIdx-1
  plot(estimatedPoses(n,1),estimatedPoses(n,2), 'ro');
  drawnow;
  hold on;
end
xlabel('x', Interpreter='latex');
ylabel('y', Interpreter='latex');
hold off;
 %% plot occupancy map
  figure;
  show(occMap);
  hold on;
  plot(estimatedPoses(:,1),estimatedPoses(:,2));

  plot(carPoses(:,2),carPoses(:,3));
  plot(trajectory(:,2),trajectory(:,3));
  legend("lidar","odometrie","trajectory");

  hold off;

end


%% Save data to file
if saveData
  % Optionally save the data to a file
  fileTimestamp = datetime("now");
  fileTimestamp.Format = "yyyymmdd_HHMMSS";
  saveFileName = sprintf('data_run_%s.mat', ...
    string(fileTimestamp));
  save(saveFileName, 'carPoses', 'odometries');
  if exist('trajectoryFig', 'var')
    save(saveFileName, 'trajectoryFig', '-append');
  end
  disp("Data saved to " + saveFileName);
end

disp("Program finished successfully.");
```

Abbildungsverzeichnis

3.1 Vergleich der geplanten Trajektorie mit der Odometrie und der Monte-Carlo-Lokalisation für die erste Kreisfahrt sowie Darstellung der Trajektorien innerhalb der verwendeten Umgebungskarte. 19
3.2 Vergleich der geplanten Trajektorie mit der Odometrie und der Monte-Carlo-Lokalisation für die zweite Kreisfahrt sowie Darstellung der Trajektorien innerhalb der verwendeten Umgebungskarte. 20
3.3 Vergleich der geplanten Trajektorie mit der Odometrie und der Monte-Carlo-Lokalisation für die erste Valley-Trajektorie sowie Darstellung der Trajektorien innerhalb der verwendeten Umgebungskarte. 20
3.4 Vergleich der geplanten Trajektorie mit der Odometrie und der Monte-Carlo-Lokalisation für die zweite Valley-Trajektorie sowie Darstellung der Trajektorien innerhalb der verwendeten Umgebungskarte. 21

Tabellenverzeichnis

3.1 Odometrisch bestimmte Positionsabweichung vor Beginn der Endpunktkorrektur 27

Codeverzeichnis

A.1 buildCircleTrajectory.m 32
A.2 buildStraightTrajectory.m 33
A.3 buildValleyTrajectory.m 34
A.4 buildEndPointCorrectionTrajectory.m 40
A.5 buildEndPointRotating2.m 41
A.6 trajectoryTo8DOF.m 42
A.7 matlab_autonomous.m 44

Literatur

[1] Frank Dellaert, Dieter Fox, Wolfram Burgard und Sebastian Thrun. „Monte Carlo Localization for Mobile Robots“. In: *Proceedings of the 1999 IEEE International Conference on Robotics and Automation (ICRA)*. Bd. 2. IEEE, 1999, S. 1322–1328. DOI: 10.1109/ROBOT.1999.772544.

[2] Niklas Schlosser. „Erstellen von Umgebungskarten auf Basis von MATLAB SLAM-Algorithmen und 3D-LiDAR-Daten“. Abgabedatum: 23.02.2024. Bachelorarbeit. Ulm: Technische Hochschule Ulm, 2024.

[3] Matthias Schmitz, Leonie Seika und Theo Nonnenmacher. *Umsetzung von SLAM-basierter Onlinekartierung und autonomer Trajektorienabfahrt in Software und Hardware am Testfahrzeug Nimbulus-E*. Bericht fortgeschrittenes Projekt in CSE. Abgabedatum: 27.11.2024. Ulm: Universität Ulm und Technische Hochschule Ulm, 2024.

[4] The MathWorks, Inc. *likelihoodFieldSensorModel – Create Likelihood Field Range Sensor Model*. MATLAB Navigation Toolbox Documentation. Accessed: 2026-09-09. URL: https://www.mathworks.com/help/nav/ref/likelihoodfieldsensormodel.html.

[5] The MathWorks, Inc. *monteCarloLocalization – Localize Robot Using Range Sensor Data and Map*. MATLAB Navigation Toolbox Documentation. Accessed: 2026-09-09. URL: https://www.mathworks.com/help/nav/ref/montecarlolocalization-system-object.html.

[6] The MathWorks, Inc. *pc2scan – Convert 3-D Point Cloud into 2-D Lidar Scan*. MATLAB Lidar Toolbox Documentation. Accessed: 2026-09-11. URL: https://www.mathworks.com/help/lidar/ref/pc2scan.html.

[7] Sebastian Thrun, Wolfram Burgard und Dieter Fox. *Probabilistic Robotics*. Cambridge, Massachusetts: MIT Press, 2005.